CHALMERS
UNIVERSITY OF TECHNOLOGY

Human Motion Replay on a Simulated Humanoid Robot Using Pose Estimation

Using a Neural Network to Translate Human Pose Estimation Results to Humanoid Robot Motion

Master's thesis in Systems, Control and Mechatronics

TOVE CASPARSSON
SIYU YI

DEPARTMENT OF ELECTRICAL ENGINEERING

CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2026
www.chalmers.se

Master's thesis 2026

Human Motion Replay on a Simulated Humanoid Robot Using Pose Estimation

Using a Neural Network to Translate Human Pose Estimation Results to Humanoid Robot Motion

TOVE CASPARSSON
SIYU YI

CHALMERS
UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering
Division of Systems and Control
Name of research group (if applicable)
Chalmers University of Technology
Gothenburg, Sweden 2026

Human Motion Replay on a Simulated Humanoid Robot Using Pose Estimation
Using a Neural Network to Translate Human Pose Estimation Results to Humanoid Robot Motion
TOVE CASPARSSON
SIYU YI

Supervisor: Hamid Ebadi, Infotiv AB, and Erik Brorsson, Department of Electrical Engineering, Chalmers University of Technology
Examiner: Knut Åkesson, Department of Electrical Engineering, Chalmers University of Technology

Master's Thesis 2026
Department of Electrical Engineering
Division of Systems and Control
Chalmers University of Technology
SE-412 96 Gothenburg
Telephone +46 31 772 1000

Typeset in LaTeX
Printed by Chalmers Reproservice
Gothenburg, Sweden 2026

Human Motion Replay on a Simulated Humanoid Robot Using Pose Estimation
Using a Neural Network to Translate Human Pose Estimation Results to Humanoid Robot Motion
TOVE CASPARSSON
SIYU YI
Department of Electrical Engineering
Chalmers University of Technology

Abstract

The imitation of human movements by humanoid robots often relies on specialized equipment, access to motion capture databases, or prior knowledge of the robot structure. This thesis, made in collaboration with Infotiv AB, presents a modular system for translating human poses to humanoid robot movements, utilizing a neural network, pose estimation and a simulated humanoid. To achieve this, a suitable simulation environment was set up and a dataset generation system was developed to collect pairs of pose estimation results and corresponding humanoid joint configurations. Through comparing different potential methods for mapping pose estimation results to joint angles, neural networks and symbolic regression were selected as potential solutions. Using the dataset generation system to collect a training dataset, the performance of a neural network and symbolic regression model was compared, with the neural network having the most promising metrics. The result of the humanoid motion predicted by the neural network from human poses showed that the pose-to-motion solution is possible, though perfect replication was not achieved. The pose and motion data collected by the dataset generation system consisted of perfectly matched pairs, but the pose data was noisy, which likely affected the neural network's performance. However, the final system is a promising foundation for a self-reliant human movement imitation system.

Keywords: Neural networks, humanoid, pose estimation, simulation, motion imitation, pose mapping.

Acknowledgements

We want to thank Infotiv AB for the opportunity to carry out this master's thesis project. We are especially grateful to our supervisor Hamid Ebadi at Infotiv AB for his guidance and support throughout the project. His technical insight and feedback during the process were essential for the work to progress smoothly.

We would also like to thank our supervisor, Erik Brorsson, and our examiner, Knut Åkesson, at Chalmers for their valuable insights and helpful feedback during the thesis.

Finally, we want to thank our family and friends for their support and encouragement throughour our studies.

Tove Casparsson and Siyu Yi, Gothenburg, January 2026

Contents

List of Figures

List of Tables

1
Introduction

This chapter introduces the project background, objectives, and related work.

1.1 Background

This project, part of the ARTWORK (The smart and connected worker) research initiative, aims to develop a comprehensive simulation framework using the Gazebo simulator for testing and validating deep learning-based safety monitoring systems. Based on the existing SIMLAN environment from Infotiv [1], the focus is on creating a 3D simulation environment that enables privacy-preserved testing of various safety scenarios, including accident detection, ergonomic risk assessment, and safety protocol compliance. Using a simulator also makes it possible to get quick feedback on changes, and implement complicated or dangerous scenarios safely.
A potential use case for the SIMLAN environment is using pose estimation to detect falls or ergonomic risks. To do this, a simulated human model is needed that can move to different positions to be evaluated. This project aims to develop a system for translating human poses to humanoid robot motion in a simulated environment. By using pose estimation software to detect the pose landmarks of a human from a camera image, the human pose is mapped to the corresponding joint configuration of a humanoid in Gazebo. The humanoid then executes the movement to replicate the pose. This project provides a foundation for the SIMLAN project to leverage pose estimation for workplace incident detection, by providing a method where a humanoid can replicate a desired pose.
The code for this project is available on GitHub[1].

1.2 Objective

The overall objective of this project is to build and train a machine learning model that translates from human pose estimation landmarks to humanoid robot motion in a simulated environment. In order to achieve this, a simulation environment and a dataset generation pipeline are set up to collect training data, after which the machine learning model is trained and evaluated.
The specific research questions investigated in this thesis are as follows:

- How can an automated pipeline be developed using ROS 2 and Gazebo to create a synchronized dataset for motion replay?

[1] `https://github.com/infotiv-research/humanoid-mocap`

We address this by developing a simulation-to-data script that captures synchronized human model joint states and camera-based pose landmarks, detailed in Chapter 4.

- Which mathematical modeling approach—analytical, symbolic regression, or neural networks—provides the most effective mapping from human poses to robot joint angles?
  A comparison of the three methods is presented in Chapter 5, and in Chapter 6 the machine learning methods are implemented and compared based on their performance.
- How effective is our neural network at creating smooth and realistic robot motions?
  We test the trained model in the Gazebo simulator and evaluate the joint movement quality (see Chapter 6).

1.3 Related Work

Human motion imitation in humanoid robots can be achieved through retargeting human motion capture data to joint movements of a humanoid robot. This generally includes retargeting human movements by mapping motion capture data to feasible robot movements [2], and using machine learning to train a whole-body controller for the robot, enabling either replay of motion capture data [3], or for teleoperation by a human using a motion capture suit [4]. In [5], neural networks were trained to learn a mapping between motion capture sensors and corresponding robot joint actuators for each degree of freedom on the robot. One major limitation of this approach is reliance on motion capture systems, which are costly and not easily accessible.
Another option for motion imitation without the need for motion capture systems is to use RGB cameras and pose estimation technologies. While still training a control policy on a motion capture dataset, an RBG camera and pose estimation were used in [6] to have a humanoid robot mimic human motions, and in [7] and [8] for both mimicking and autonomously performing motions learned from human demonstration.
OpenVICO [9] is a toolkit for skeleton tracking and human-robot collaboration, using Gazebo Classic as a simulation environment and OpenPose for pose estimation. Among other applications, it proposes methods to create large datasets of human pose estimation data for different actions and fusing pose estimation results from a multi-camera system. OpenVICO is, however, built on ROS 1 and Gazebo Classic which have both reached end of life.

Another method is to directly set the robot joint angles using joint angles calculated from a human pose as input to robot actuators, using an RGB-D camera [10], or an RGB camera and pose estimation [11], [12]. However, this approach requires some joint angle calculations and knowledge of the robot structure.

Our approach is to train a neural network using the joint positions read from a simulated humanoid robot, and the pose detection result captured in that specific

joint configuration. By doing so, there is no need to use motion capture data, but instead the system learns to reproduce a pose detection result by moving to the corresponding joint positions.

1.4 Paper outline

This thesis is structured as follows. Chapter 2 introduces the overall pipeline created in this project and some necessary theoretical background. Chapter 3 presents the simulation environment setup and humanoid model used in the project. Chapter 4 describes the system for creating a dataset to be used to train machine learning models. Chapter 5 introduces and discusses potential methods for mapping pose estimation data to humanoid joint positions. Chapter 6 presents the results of the machine learning models and discusses the full system, potential improvements, and future work. Finally, Chapter 7 summarizes the thesis project.

2

System Overview

In this chapter, a high-level overview of the developed motion replay system is provided. The goal of the system is to translate human poses into corresponding humanoid robot joint angles in a simulated environment.

2.1 Overall Pipeline

The system is designed as a modular pipeline to translate human poses into humanoid robot joint states. As shown in Figure 2.1, the process is divided into three primary stages: perception, where human landmarks are extracted; translation, where the mapping algorithms compute joint angles; and execution, where the robot performs the motion in a physics-based simulation.

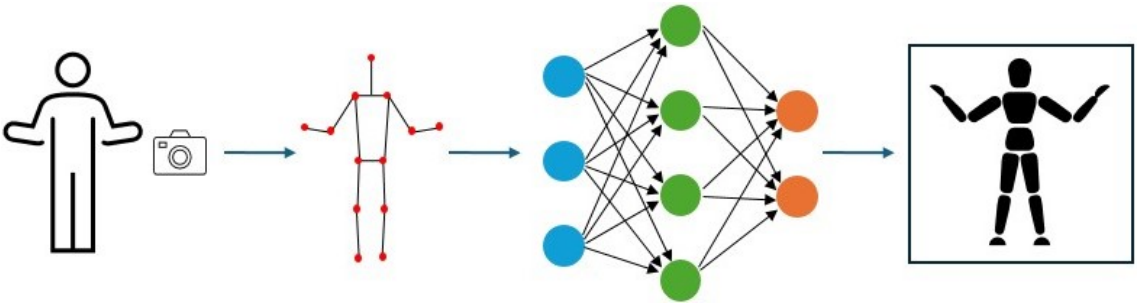


Figure 2.1: Overview of the full system. From a real-life picture, pose landmarks are detected and given as input to a neural network, which performs pose-to-motion mapping. The output joint angles are then given to the simulated humanoid, which performs the desired motion.

Throughout the project, ease of use was taken into consideration both for the final system and to enable modifications to the system if necessary. The pose estimation module takes a 2D RGB picture and estimates the coordinates of each landmark in 3D, making it possible to use with only a webcam or phone camera. The neural network then predicts the joint angles and the humanoid robot would need to have in order to produce the same pose estimation result, which are then given to the humanoid robot to replicate the desired pose.

2.2 Dataset Generation Strategy

Training a neural network requires a large training dataset, and thus a dataset generation pipeline was created. By moving the simulated humanoid to different positions and capturing the pose estimation result and joint angles in each, a dataset consisting of matching pairs of poses and joint angles was created. Using this dataset to train the neural network, the training process is self-contained and does not rely on access to external datasets. Thus, it is possible to swap out the humanoid robot and the pose estimation model, generate a new training dataset, and train a new neural network with only minor code edits.

The ability to generate new training datasets and the modular design makes the system flexible and easy to modify. For example, it is possible to swap the neural network model for another with only minor code edits, provided that the input and output remain the same. It is also possible to swap out the humanoid model and the pose estimation model, generate a new training dataset, and re-train the neural network model to predict joint angles for the new humanoid based on the new pose estimation result.

2.3 Neural Networks

Neural networks are particularly adept at capturing complex, non-linear mappings from input to output, addressing the challenge of converting human pose to robot motion across differing kinematic structures.

2.3.1 Artificial Neuron Models and Multilayer Perceptrons

An artificial neuron receives multiple input signals (x_i), each associated with a weight (w_i) that determines its relative importance. As shown in Equation 2.1, the neuron computes a weighted sum, adds a bias (b), and passes the result through an activation function (f):

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(\mathbf{w}^T\mathbf{x} + b) \tag{2.1}$$

While single-layer perceptrons are limited to linearly separable patterns, multilayer perceptrons (MLPs) incorporate hidden layers to facilitate complex transformations between human pose data and robot joint configurations.

2.3.2 Activation Functions and Forward Propagation

Activation functions introduce non-linearity into the network [13]. Some common functions used in this field are:

- Sigmoid and Tanh: Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$ maps values to (0,1), while Tanh maps to (-1,1). Both are historically significant but can suffer from vanishing gradient problems.

- ReLU: $\text{ReLU}(z) = \max(0, z)$ is widely used due to its computational efficiency in mitigating vanishing gradients.
- GELU: $\text{GELU}(z) = z\Phi(z)$ weights inputs by their value multiplied by the Gaussian cumulative distribution function $\Phi(z)$ [14]. Unlike ReLU, GELU provides a smooth, probabilistic weighting.

Information flows from the input layer through hidden layers to the output via forward propagation. For a network with L layers, this is formalized as:

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}, \quad a^{[l]} = g^{[l]}(z^{[l]}) \tag{2.2}$$

where $W^{[l]}$ and $b^{[l]}$ are the weight matrix and bias vector, and $a^{[L]}$ represents the final prediction.

2.3.3 Loss Functions and Backpropagation

To quantify the discrepancy between predicted outputs ($\hat{y}_i$) and actual targets (y_i), we employ the Mean Squared Error (MSE) loss:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2 \tag{2.3}$$

Backpropagation [15] calculates the gradients of the loss function with respect to network parameters using the chain rule, allowing for recursive updates starting from the output layer.

2.3.4 Evaluation Metrics

To provide a comprehensive evaluation beyond optimization, the following metrics are used:

- MAE: Evaluates the average absolute deviation in joint angles.
- R^2 Score: Measures how well the predictions approximate the true variance in the data; a score of 1 indicates a perfect fit [16].
- Cosine Similarity: Measures the directional alignment between the predicted and ground-truth motion vectors, ensuring the orientation of the replayed motion is preserved:

$$\text{CosineSim} = \frac{\hat{\mathbf{y}} \cdot \mathbf{y}}{\|\hat{\mathbf{y}}\| \, \|\mathbf{y}\|} \tag{2.4}$$

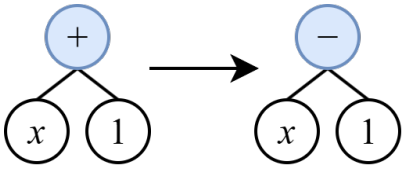
2.4 Symbolic regression

Symbolic regression is a machine learning approach in which the aim is to learn a mathematical formula (symbolic expression) that describes the relationship between given input and output data [17]. This is done by exploring combinations of simple mathematical operations, such as addition, multiplication, and trigonometric functions, and evaluating their fitness on the dataset to generate the final model, with the aim that the model should be interpretable by humans.

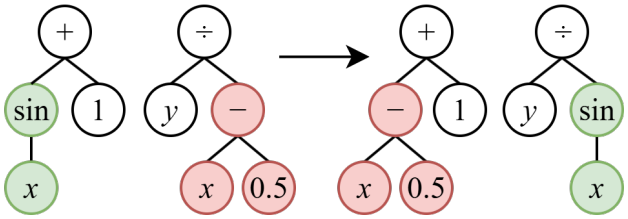
A common method for symbolic regression is genetic programming [18], an evolutionary algorithm that uses operations inspired by natural selection to evolve an initial set of expressions. In genetic programming, mathematical expressions are represented as a tree structure, where each node is a binary or unary operator from a set of possible operators, and each leaf is a variable or a constant. From an initial population of randomly generated trees, the population is evolved through selections, mutations, and crossovers to find the best fitting function. First, the fitness of the entire population is evaluated. If the fitness criterion is not reached, the population is evolved as follows:

- Selection: individuals are selected to serve as parents for the next generation, with individuals with higher fitness having a higher probability of being selected.
- Mutation: a random mutation is applied to the selected individual, replacing an existing part of the tree.
- Crossover: a random sub-tree is swapped between two selected individuals.

When the fitness criterion is reached, the best individual is presented as the result. In this way, the algorithm behaves similar to evolution, where individuals with higher fitness replace those with lower fitness.



(a) Mutation on one expression tree.



(b) Crossover between two expression trees.

Figure 2.2: Illustrations of the mutation and crossover operations in genetic programming. Note that the expressions are only for illustration purposes and unrelated to the project.

3
Simulation Platform and Modeling

This chapter details the setup of the simulation environment, including the configuration of the Gazebo simulation world, the selection of the simulated human model, and the implementation of the pose capture system.

3.1 Simulation Infrastructure

The first step in this project was to build a suitable simulation environment. The environment must contain a suitable human model whose movement can be controlled, a camera to capture images of the human model, and a pose estimation module that processes the images to detect the pose landmarks. The system must also be compatible with the larger SIMLAN simulation environment, so that it can be easily integrated.
To meet these requirements and handle the complex communication between different modules, a robust middleware and physics engine are essential. In this project, the Robot Operating System 2 (ROS 2) and the Gazebo Simulator were selected as the core software framework, with MoveIt 2 being used for control.

3.1.1 Robot Operating System 2 and Gazebo Simulator

ROS 2 is an open-source software development kit for developing robotics applications [19], and the successor of ROS 1. It provides a standardized format for sending messages within the system, visualization through RViz, and an extensive library of packages for different functionality in robotics applications such as drivers and navigation tools. A ROS 2 system is built as a network of nodes, where each node performs some unique kind of computation and can communicate with other nodes, allowing a modular way of constructing a whole system.

Nodes can communicate through publish-subscribe topics for continuous data streams, request-response services where one node requests another node to perform some computation and return the result, and actions, made for longer-running processes that consist of a request, response, and periodic feedback.

3.1.2 Gazebo Simulator

Gazebo Simulator (Gazebo) [20] is used as the physics engine to provide a realistic 3D environment. It is an open source robotics simulator that has many features beyond

visualization, such as physics engines and sensor plugins. Gazebo is a standalone simulator, but it can easily be integrated with a ROS 2 system using the`ros_gz`[1] package, enabling the use of ROS 2 to launch and communicate with the simulation world.

The simulation world in Gazebo is defined in an SDFormat (SDF) file, which defines properties such as the physics engine, graphical user interface settings, which plugins to load, and objects in the world. Gazebo also supports spawning URDF robots, both natively and through ROS 2 using the `ros_gz` plugin. For a URDF robot to function properly in Gazebo, all links need to have inertia specified, and to integrate ROS 2 control with Gazebo the `gz_ros2_control`[2] is used.

In this project, Gazebo Fortress was used, as it is the recommended version to use with ROS 2 Humble.

3.1.3 MoveIt 2

MoveIt is an open-source robotic motion planning and manipulation framework for ROS [21], with MoveIt 2 being the successor compatible with ROS 2 and the version used in this project [22]. MoveIt functionality includes motion planning, collision detection, and forward and inverse kinematics in a plugin-based architecture, allowing the user to customize according to their needs. New robots can be configured to be used with MoveIt through the MoveIt Setup Assistant, a graphical user interface which automatically generates the configuration package for MoveIt after the user has performed the initial configuration of a URDF or Xacro robot step by step.

The key ROS 2 node provided by MoveIt 2 is the `move_group` node, which ties together the plugins for MoveIt functionality and provides ROS interfaces for the user. The `move_group` handles robot communication, motion planning, kinematics solvers, collision detection, and trajectory execution.

In this project, the unofficial `pymoveit2` package [23] was used as the Python interface for MoveIt 2, as the official MoveIt 2 Python package `moveit_py` [24] was not available for ROS 2 Humble.

3.2 Humanoid Robot Configuration

Robots in ROS are defined in a Unified Robot Description Format (URDF), which is an XML format for describing a robot. The URDF contains information about the robot, such as links, joints, collision boxes, controllers, and sensors. A URDF can also be generated from an XML Macro (Xacro) file, in which case the expressions in the Xacro file are expanded from a more readable format.

[1] `https://github.com/gazebosim/ros_gz/tree/humble`
[2] `https://github.com/ros-controls/gz_ros2_control`

The human model used in this project is based on the "Human subject with meshes" URDF model from Robotology[3]. This model was chosen as it was both human-like and ready to use in Gazebo.

The model does not have any joint controllers defined in the URDF, so when spawning it in Gazebo as is it would simply collapse. In order to keep the humanoid upright without needing to create a controller setup for balancing, a support arm consisting of a base link and four arm links was created in Xacro format, which can be modified easily. The first joint is fixed to put the humanoid slightly above the ground, followed by three revolute joints along the (x, y, z) axes in order from the bottom. Using another Xacro file to combine the support arm and humanoid, the last arm link was attached to the humanoid pelvis link with a fixed joint at a 20 cm distance from the support arm, allowing the humanoid to lean and twist freely without falling over.

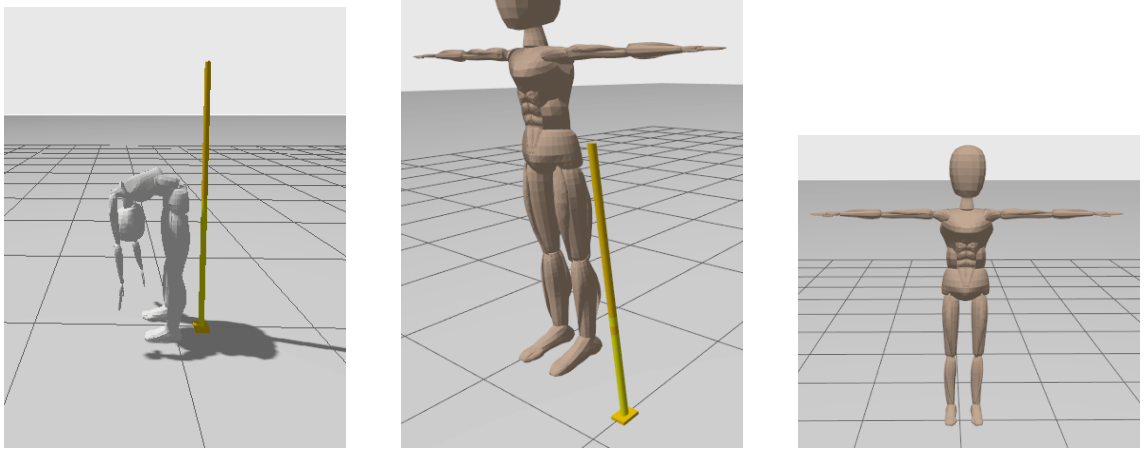
After adding the support arm, there are a total of 47 movable joints, 44 from the humanoid and 3 from the arm. The humanoid with an early version of the support arm without revolute joints can be seen in Figure 3.1a. As the humanoid does not yet have joint controllers defined, it is kept from falling only by the support arm.

To enable joint control, a MoveIt configuration package containing both the support arm and humanoid as one planning group was created using the MoveIt Setup Assistant.Using the Assistant, control interfaces for each joint, ROS 2 controllers, and corresponding MoveIt controllers were automatically generated. Finally, the necessary tags for the `gz_ros2_control` package were added manually to the MoveIt package, enabling control of the joints in the Gazebo world.
Some modification to the launch files in the MoveIt package was required for the system to launch properly. For the MoveIt controller manager to communicate with the robot, the robot needs to publish its controller interface. However, Gazebo started too slowly, so the robot did not publish a controller interface to connect to before the MoveIt controller manager would time out. This was solved by separating the Gazebo and controller launch from the rest of the launch file.
In addition to the supporting arm and MoveIt configuration package, the humanoid model URDF was also slightly modified to work well in this project. The following modifications were made to the URDF:

- Links in the URDF representing sensors that were not used in this project and their corresponding joints were removed.
- The colour of all links was changed, as the humanoid initially appeared a similar grey to the background in Gazebo.
- In an URDF, revolute joints can only have a single rotational axis. To emulate joints with multiple directions of movement, for example the shoulder joint, multiple invisible links with different rotational axes for the joints are stacked together. Inertia was added to these links, so that Gazebo could handle them.
- The weight of all links was reduced by dividing the original weight by 100, as the robot was unable to move to and from some positions due to the weight of the links.

[3] `https://github.com/robotology/human-gazebo`

The modified version of the humanoid with the support arm visible can be seen in Figure 3.1b, but to avoid issues with the pose estimation, the visual and collision elements of the support arm were disabled for the final version, which can be seen in Figure 3.1c.



(a) Humanoid in Gazebo with support arm, but no joint controllers

(b) Humanoid in Gazebo with visible support arm and joint controllers

(c) Humanoid in Gazebo with support arm hidden and joint controllers

Figure 3.1: Humanoid model in Gazebo simulator, with and without the support arm visible.

3.2.1 Human model alternatives

Another option for the human model in the simulation environment was using the Gazebo native actor object[4]. The actor can be detected by simulated cameras and lidar sensors, but are not affected by physics and do not interact with other objects in the simulation. They can be animated by defining a trajectory for the actor to move along during the simulation where all the links move as one group, a skeleton animation which defines the motion of the actors' links relative to each other, or both a trajectory and skeleton animation at the same time. Gazebo supports actor skins from COLLADA files, and animations from either COLLADA or Biovision Hierarchy (BVH) files, as long as the skeleton is compatible between the skin and animation.

However, in the context of this project, there are some drawbacks to the Gazebo actor. Firstly, the actor animations need to be defined in the .sdf file that defines the simulation world before the simulation is launched, and they cannot be changed while the simulation is running. This means that every animation change would require stopping the simulation, editing the .sdf file, and restarting the simulation. Secondly, in COLLADA and BVH files it is difficult to modify the existing animation in the file itself. In both file types, the numerical animation data for each joint is

[4] `https://gazebosim.org/docs/fortress/actors/`

stored in the file, and the position of the joint is calculated from that value for each frame when running the animation. As such, editing the animation for a human model would require modifying the numerical animation data correctly for each joint in each frame, which would be extremely difficult. An attempt was made to instead modify animations from both file types using Blender[5], an open source software for 3D graphics, but this also proved to be extremely difficult and time consuming as neither of the authors have any experience with animation or Blender. There are databases of animations that could be used as-is, but that would rely on someone else having already created the desired animation.
Due to the limitations of the Gazebo actor and the difficulties with modifying animations, the URDF model was chosen instead, as it can be integrated with Gazebo and controlled from ROS 2.

3.3 Camera setup

A simulated RGB camera was added to the simulation environment, placed at coordinates $(3.5, 0.0, 2.0)$, right in front of the humanoid robot, and with a slight pitch to capture a complete view. The camera system subscribes to the Gazebo image feed via a topic that is bridged from Gazebo to ROS 2, and converts the images to OpenCV format.

3.3.1 Pose Estimation Module

Human pose estimation is a computer vision task that involves detecting key anatomical landmarks on human bodies in images or videos. By identifying joints such as shoulders, elbows, wrists, hips, knees, and ankles, the system estimates the position of these landmarks in relation to each other to estimate the full body pose.
From a technological development perspective, pose estimation has evolved from traditional methods to deep learning approaches [25].

There are several frameworks for human pose estimation, and each is appropriate for various situations. Firstly, OpenPose[26] is a human pose estimation model based on CNN and Part Affinity Fields by Carnegie Mellon University. It is also one of the earliest popular methods, because of its strong performance in multi-person detection scenario such as sports analytics, human–computer interaction.

MMPose [27] is an open-source pose estimation toolbox based on PyTorch and is a member of the OpenMMLab project. It supports widely in different fields including both 2D and 3D human, hand, and face keypoint detection.

MediaPipe Pose [28], on the other hand, focuses on efficiency and real-time performance, capable of running on mobile and embedded devices. It's ability to return low-latency results have made it popular in interactive use-cases like fitness coaching, augmented reality, and workplace monitoring. MediaPipe also has the strength

[5] `https://www.blender.org/`

of lightweight character and the ease of integration in a real-time robotic simulation environment.
For the pose estimation module, a solution that could predict a 3D pose from 2D RBG images was required, while also being computationally efficient and simple to integrate in a ROS 2 framework. The selected solution for pose estimation was Google MediaPipe, which predicts body landmarks in 3D using the midpoint of the hips as the depth origin. MediaPipe can also be integrated with ROS 2, for example in the ROS4HRI project[6].
To detect the landmarks of the humanoid, the OpenCV format images are processed through the MediaPipe Pose model to identify the pose of the humanoid in the frame. If a pose is detected, the image with is displayed with pose detection markers and connections for reference.

3.4 Final Simulation Environment

The final simulation environment is a Gazebo environment using the Gazebo world `empty.sdf` as a base, containing a humanoid robot and a simulated RGB camera. ROS 2 is used for communication with the simulation, MoveIt 2 for motion planning and control of the humanoid, and Google MediaPipe is the pose estimation module processing the image feed from the camera. A diagram of the simulation environment is given in figure 3.2.

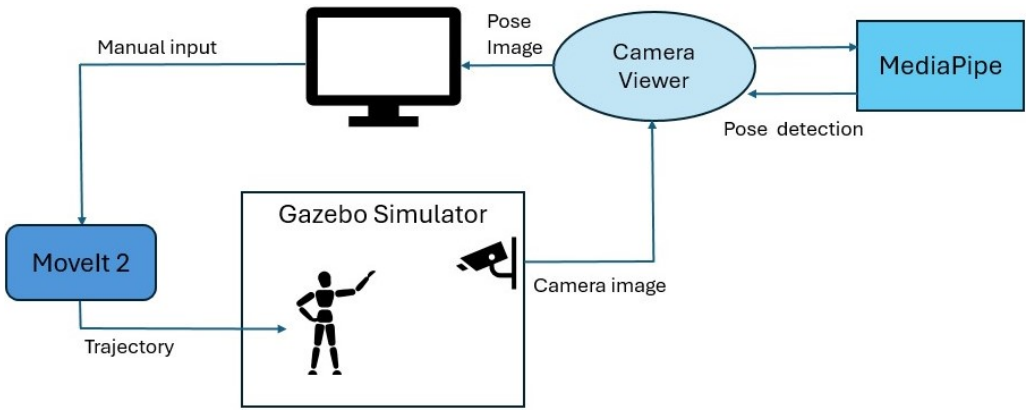


Figure 3.2: Diagram of the simulation environment, including Gazebo, MoveIt 2, and the camera viewer node, and the communication within the system.

[6] `https://wiki.ros.org/hri`

4
Dataset Collection

This chapter describes the system for generating a training dataset to be used for machine learning. An overview of the modifications made to the simulation environment is given, followed by a more detailed explanation of each part. The mechanism for ensuring that the dataset does not contain mismatches is described. Finally, the procedure for running the full dataset generation system is described.

4.1 Dataset generation

In order to train an ML model, a large dataset of inputs and corresponding outputs was needed. In the context of this project, the system needed learn to map pose detection data to the ROS 2 joint positions necessary to move to the same pose. To avoid confusion, "motion" refers to the ROS 2 joint position and movement in the simulation, and "pose" refers to the pose detection data captured by MediaPipe.
For the generation of a dataset to be feasible, the process of performing a motion and capturing the pose and motion data must be automated. Using the MoveIt 2 GUI to move the humanoid would require manual input for every new motion, and sending joint commands using ROS 2 topics would require prior knowledge and the definition of a large number of valid motions.
To generate a dataset of pose-motion pairs, the simulation environment described in Chapter 3 was expanded with a node to generate random motions for the humanoid, and communication with the camera viewer node was set up that ensured data synchronization, with each pose-motion pair being given a unique numerical ID, here referred to as the "motion ID". Figure 4.1 illustrates the desired pipeline for creating a dataset of matching pose-motion pairs.

Figure 4.1: Illustration of the dataset generation pipeline, where "Gz" refers to Gazebo and "MP" refers to MediaPipe.

4.1.1 Random motion generation strategy

To generate random motions, a random motion planner node was created to handle assigning the motion ID, setting random values as target positions to each joint in

the humanoid, communication with MoveIt 2, and saving the motion data.
First, the motion ID was assigned to the current motion as a random number between 1 and 9999999999. Then, each joint was assigned a random value as target position. The maximum amplitude of the motion, A, was given by the user, and there are two options for how it was used. The first option was the offset option, where the target position was set by adding an offset to the current joint position, and the second option was the normal absolute value option, which directly assigns a new target position to the joint.
The offset option draws the offset value from a uniform distribution

$$X \sim \mathcal{U}(-A,\, A) \tag{4.1}$$
$$pos_{target} = pos_{current} + X \tag{4.2}$$

And the normal absolute value option draws a new value from a Gaussian distribution

$$X \sim \mathcal{N}\left(0,\, \left(\frac{A}{8}\right)^2\right) \tag{4.3}$$
$$pos_{target} = X \tag{4.4}$$

The motion from the current positions to the target positions was then planned and executed by MoveIt 2, using the pymoveit2 command `move_to_configuration`. If the planning or execution of the motion fails, the process would start over with a new motion and motion ID. If the motion was executed successfully, a request was sent to the camera viewer node with the current motion ID to capture the pose. If the camera viewer successfully captured the pose, the current joint positions were saved as the motion data, using the motion ID as the file name. The data was saved in JSON format, with joint name and position for each of the 47 joints.
The random motion planner node was also set up to handle motion replay, in which case a file containing motion data would be given when starting the node. In that case, the pose detector node would not be started, and the motion data from the file was simply sent to MoveIt 2 to plan and execute.

4.1.2 Camera viewer node

The camera viewer node subscribes to the Gazebo camera feed. If the random motion planner node sends a request to capture the current pose, the current image is passed to MediaPipe Pose for pose estimation. If the pose landmarks are detected, the landmark coordinates are saved and images of the current pose estimation with and without the landmarks are saved using the motion ID as file name identifier. Once the pose data are saved, the node returns True to the random motion planner through the service. If MediaPipe fails to detect pose landmarks, the camera viewer returns False to the random motion planner and does not save any data. The landmark data are saved in JSON format, with the landmark index and (x, y, z) coordinates for each of the 33 landmarks.

4.1.3 Joint state and pose data synchronization mechanism

For the generated dataset to be useful, the pose-motion pairs need to refer to the same robot position. In other words, the random motion planner and camera viewer must wait for each other, so that the motion is finished before the pose estimation is performed, and no new motion is started before the pose estimation is done for the current one.

To ensure synchronization between the random motion planner and the camera viewer, a ROS 2 service was used for communication between the nodes, with the random motion planner being the service client and the camera viewer the service server. Initially, the setup was based on the random motion planner publishing a message with the motion ID on a topic where the pose detector was a subscriber, but this caused synchronization issues as the random motion generator did not know if the pose detector had successfully captured the pose before moving on.

When a motion has been successfully executed, the random motion planner node sends a request containing the current motion ID to the camera viewer node to capture the pose. This call is blocking, so the random motion planner will not continue until a response is received from the camera viewer, and the camera viewer does not perform pose estimation unless a request is received. To ensure that motion data is not saved if the pose estimation was not successful, the motion data is only saved after the camera viewer returns True, otherwise the motion data is discarded.

4.2 Full dataset generation procedure

For convenience, a control script was created to run the system, which starts all necessary processes and restarts everything every 10 minutes, as parts of the system would occasionally crash or become unresponsive. When running the full dataset generation system, the simulation environment is launched first, followed by the camera viewer and random motion planner nodes. As Gazebo can be somewhat slow to start, a five second delay is set between the start of the simulator, pose detector, and random motion planner to avoid issues where nodes try to contact other nodes that are not running yet. Figure 4.2 provides an overview of the full dataset generation system, including nodes and communication.

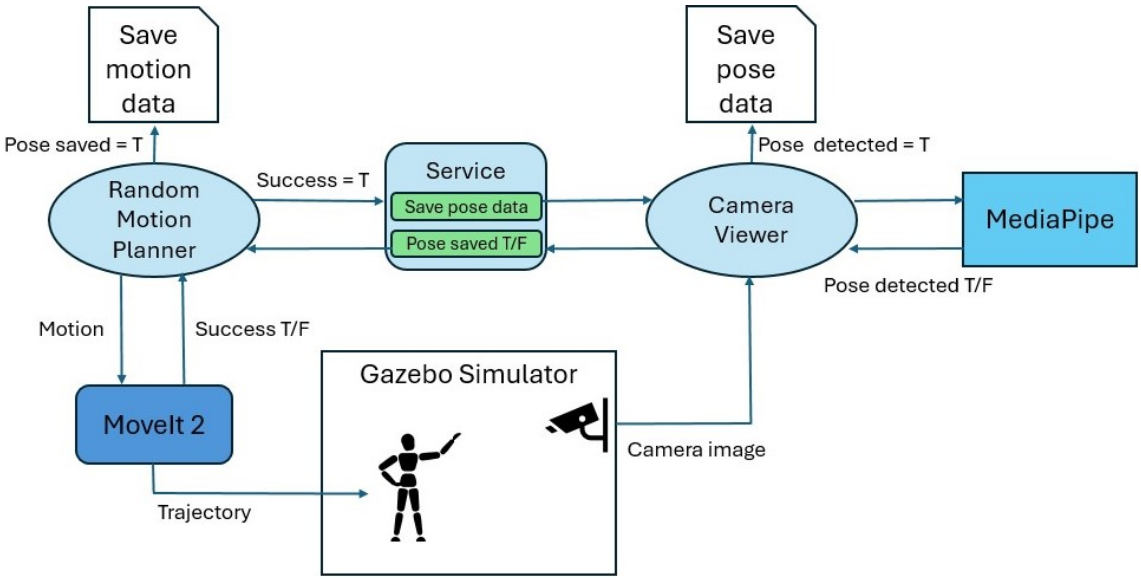


Figure 4.2: Diagram of the dataset generation system, including Gazebo, the random motion planner and camera viewer nodes, and the communication between them.

To obtain some diversity in the dataset with respect to the humanoid position in the camera frame, the spawn position of the humanoid in the Gazebo world was set to be random from the Gazebo world origin with $-1 \leq x \leq 1$ (front to back) and $-2 \leq y \leq 2$ (side to side). The limits of the spawn position were set to be close enough for MediaPipe to detect landmarks, and distant and centered enough that the humanoid or its limbs would not be out of frame.
Full details on how to run the system using a control script are available on the project GitHub.

5

Comparison of Joint Angle Learning Methods

In this chapter, potential methods for translating the pose estimation landmark coordinates to joint positions in ROS 2 are examined. Three approaches are discussed: calculating the joint angles from spatial coordinates using mathematical analysis, training a neural network to predict the mappings, and finding the mathematical expressions using symbolic regression.

5.1 Mapping pose data to joint positions

For the simulated humanoid to replicate a motion based on a pose estimation result, a method was needed to map pose estimation results to joint positions. More specifically, a MediaPipe pose estimation result should be converted to ROS 2 joint positions that can be replayed by the simulated humanoid.

Inferring relationships between input and output data to perform a task is the essence of machine learning. The dataset generation described in Chapter 4 made pose-motion pairs describing the pose detection result of different motions available for analysis. However, using mathematical analysis to find the mapping from the detected landmark coordinates to the humanoid's joint angles could also be a solution.

Therefore, in this chapter, three different methods were analyzed for this task. First, the analytical approach is introduced, that uses mathematical equations to directly calculate joint angles from spatial coordinates. Second, machine learning methods that are suitable for handling large-scale and complex mappings were examined, especially deep learning. As a comparison to deep learning, symbolic regression is discussed, which aims to find interpretable mathematical expressions from data.

By breaking down and evaluating these methods through both theoretical analysis and practical experiments, we aim to find the most suitable solution for converting human pose data into humanoid robot motion.

For comparing the methods, some criteria were established for selecting the best method. Ideally, the method would be "plug-and-play", so that a user would not have to do more than start the simulation system and provide an image for pose estimation. The motion replay should be accurate to the desired pose, and the method must be resistant to noise in the pose estimation data.

5.2 Analytical method

The analytical approach was to compute the joint angles directly from geometric relationships, using known kinematic equations of the robot. In this study, Matlab was used to simulate the kinematics of one upper limb, focusing on one segment from the shoulder joint to the elbow joint. Here, the elbow was treated as the end-effector of this segment.
Given the 3D coordinates of the elbow, the corresponding angles of the shoulder joint could be calculated. Specifically, the rotation around the x axis (shoulder abduction/adduction) and the rotation around the y axis (shoulder flexion/extension) are illustrated in Figure 5.1 and the equations for calculating the joint angles are shown in Equations 5.1 and 5.2 respectively.

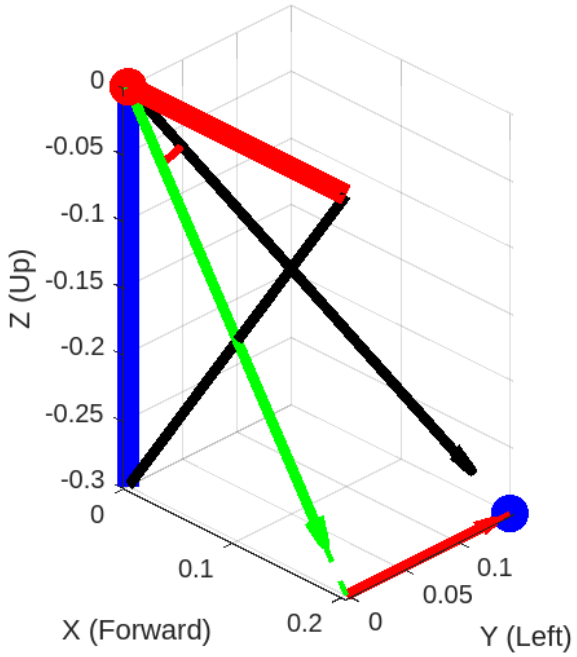


Figure 5.1: Joint angle calculation of rotation around the x and y axes using Matlab.

$$\theta_1 = \arctan\left(\frac{\Delta y_1}{\sqrt{\Delta x_1^2 + \Delta z_1^2}}\right) \rightarrow \text{jLeftShoulder\_rotx} \tag{5.1}$$

$$\theta_2 = \arctan\left(\frac{\Delta x_1}{\Delta z_1}\right) \rightarrow \text{jLeftShoulder\_roty} \tag{5.2}$$

Where θ_1 is the azimuth angle and θ_2 is the elevation angle.
Although the analytical method is highly accurate, it is pretty idealized, and also relies on a fully known kinematic humanoid model. Obtaining an accurate motion by directly calculating the joint angles from landmark coordinates is also not resistant to inaccuracies in the pose estimation data. Furthermore, with a large number of joints, in this case 47, the method becomes complex and computationally intensive.

5.3 Deep Learning method

Given the limitations of solving each joint position analytically, machine learning could be a viable alternative. Neural networks are capable of directly learning non-linear mappings in high-dimensional space from data. Therefore, a deep learning pipeline was designed that covers data preprocessing, model architecture, and training, aiming for pose-to-motion translation. This section presents the implemented deep learning pipeline, including data preprocessing, and model design and training. The code for the data preprocessing, model architectures, and training loop can be seen in Appendix A.

5.3.1 Data Preprocessing

The pose-motion dataset consisted of two parts. The motion data representing the ROS 2 joint position data used to instruct the system to move the humanoid, and the pose estimation result obtained from Google MediaPipe through images of finished motions. The JSON format used to save both pose and motion data was appropriate for human readability, but for the purpose of machine learning, the data were extracted into Numpy arrays.
As the pose data can vary significantly with different global positions, a function to normalize pose data using a reference landmark at the left hip was created, in order to reduce the impact of global position variations.

5.3.2 Model Architecture

Two model architectures were tested: a simple feed-forward neural network and an improved neural network. A simple feedforward neural network, named `SimpleMotionModel`, consisting of three fully connected layers with ReLU activation functions was created as an initial model for pose-to-motion mapping. This architecture was simple enough to allow for a fast exploration, while still capturing the core non-linearity of the connection between pose and motion data. However, when tested on high-dimensional data, the simplicity was found to limit its ability to capture more complex patterns. As such, an `Improved Motion Model` was created as an enhanced version of the simple model, with the following architecture:

- **Residual Connections:** Residual connections alleviate the vanishing gradient problem. Especially in deeper networks, gradients can diminish as they propagate backward. Residual connections allow the network to "skip" layers and directly pass information forward. This design was first introduced in ResNet [29].
- **Batch Normalization:** Batch Normalization reduces the problem of internal covariate shift in neural networks by normalizing the data within each mini-batch. This technique was introduced by Ioffe and Szegedy [30]. By calculating the mean and variance of the data in a batch, the values are adjusted so that they have a similar range. This improves the stability of the optimization process and reduces sensitivity to weight initialization.
- **GELU Activation:** The GELU activation function considers the input value

relative to a Gaussian distribution, rather than gating the input based solely on its sign like ReLU, making it more effective for regression tasks like pose-to-motion mapping.

- **Xavier Initialization:** Xavier initialization was used to set the weights of the network. This ensures that the variance of outputs remains consistent across layers, and helps with avoiding exploding or vanishing gradients during training. This method was introduced by Glorot and Bengio [31].

The overall architecture of the `ImprovedMotionModel` is illustrated in Figure 5.2.

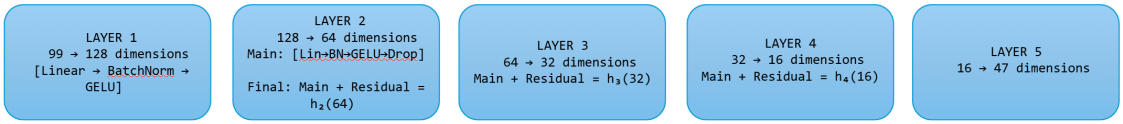


Figure 5.2: Layer-wise architecture

5.3.3 Training Process

The training process included batching, gradient clipping, learning rate scheduling, and early stopping. Validation metrics, including MSE, MAE, R^2 score, and cosine similarity, were computed to monitor performance.

5.4 Symbolic Regression using PySR

In order to evaluate the performance of the neural network, symbolic regression was considered as an alternative method. As SR aims to discover explicit mathematical relationships directly from data, it was deemed a suitable option.
PySR is an open-source tool for symbolic regression, with a Python frontend and using Julia as backend [32]. Using PySR, the user defines a model with a set of mathematical operators to be used in the symbolic regression search, and optional custom features such as maximum equation size, model selection method, loss function, and custom operators.

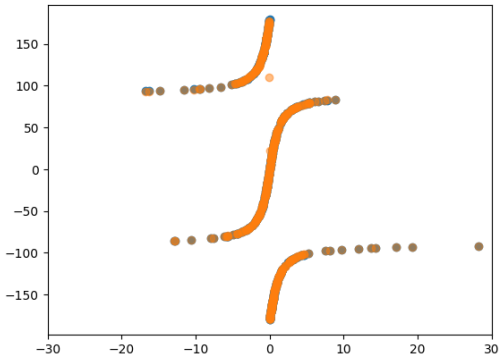
5.5 Toy example on methods

To introduce neural networks and symbolic regression using PySR, two sets of 500 data points each were generated as input data, one of (x, y) coordinates along the circumference of a circle and the other of (x, y, z) coordinates on the surface of a sphere, both with a radius of 30. These points would represent possible end-effector positions of a robot arm in 2D and 3D respectively, and the corresponding joint angles were calculated and given as the output data. To investigate resistance to noise in the methods, models were trained on the clean datasets, and with Gaussian noise added to the input coordinates. The added noise was drawn from $\mathcal{N}(0,\, 5^2)$ for both the 2D and 3D datasets.

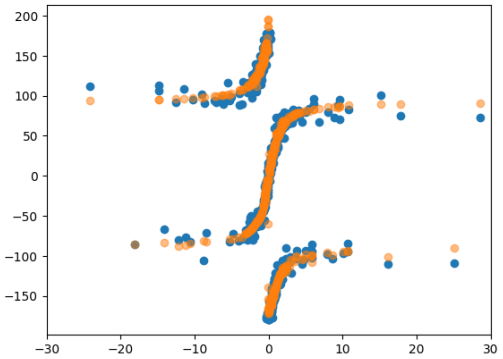
Ideally, PySR would discover the correct equation to calculate the joint angle with both clean and noisy data. Neural networks do not directly output their discovered equations, but instead the accuracy is judged by error metrics.

5.5.1 Toy example of neural network

The neural network used in this example was the `SimpleMotionModel` described in Appendix A.
In the 2D example, the predictions were very accurate for the dataset with no noise, which are plotted in Figure 5.3a. As can be seen in table 5.1a, error rates were low while explainability indicated by the R^2 score is high. For the noisy dataset, the predictions plotted in Figure 5.3b were less accurate. The error rates in Table 5.1b indicate that the model has not learned to average over the added noise, possibly due to the small underlying dataset.



(a) 2D data prediction (orange) vs. ground truth (blue) for a dataset without noise.



(b) 2D data prediction (orange) vs. ground truth (blue) for a dataset with added Gaussian noise.

Figure 5.3: Neural network 2D data prediction (orange) vs. ground truth (blue), for a 2D dataset with no noise and a dataset with added noise.

MSE Loss	0.0373
RMSE	0.1932
MAE	0.0180
R^2 Score	0.9893

(a) Error metrics for the neural network for a 2D dataset without noise.

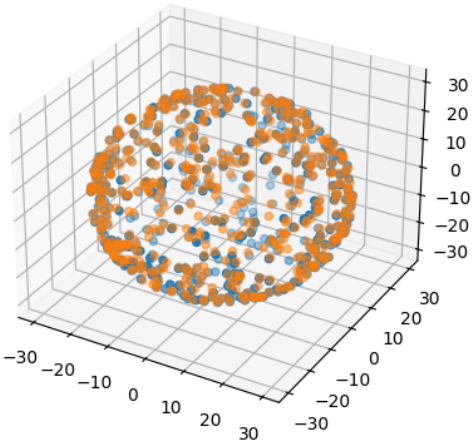
MSE Loss	0.9386
RMSE	0.9688
MAE	0.3032
R^2 Score	0.7304

(b) Error metrics for the neural network for a 2D dataset with added Gaussian noise.

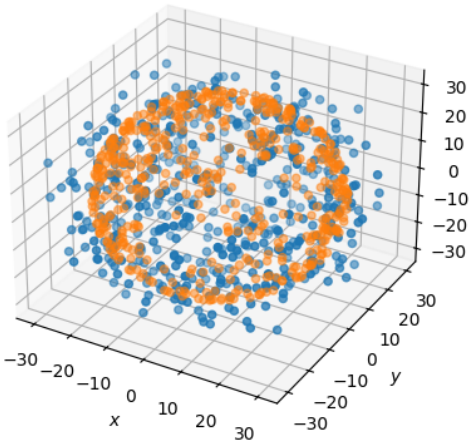
Table 5.1: Error metrics for the neural network on the 2D dataset.

For the 3D example, predictions on the dataset with no noise were again very accurate, as can be seen in Figure 5.4a and the error metrics in Table 5.2a. On the dataset with noise the predictions, see Figure 5.4b, are less accurate. The error metrics in Table 5.2b are worse than for the clean datasets, but better than the

noisy 2D example, indicating that the model was less sensitive to noise in higher dimensions. This could be explained by the increased spatial relationships between points in 3D space allowing the model to better average over the noise.



(a) Comparison of Ground Truth and Predicted 3D Positions



(b) Comparison of Noisy Ground Truth and Predicted 3D Positions

Figure 5.4: Neural network 3D Data Prediction (orange) vs. Ground Truth (blue), for a 3D dataset with noise.

MSE Loss	0.0239
RMSE	0.1547
MAE	0.0342
R^2 Score	0.9929

(a) Error metrics for the neural network for a 3D dataset without noise.

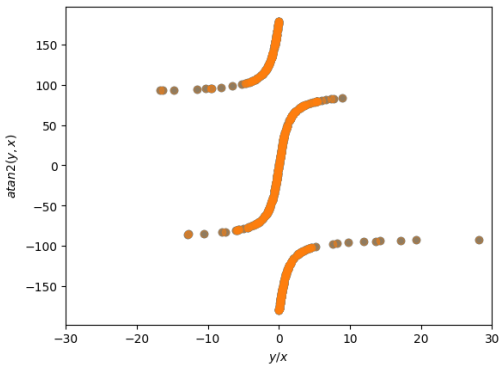
MSE Loss	0.5394
RMSE	0.7344
MAE	0.3349
R^2 Score	0.8035

(b) Error metrics for the neural network for a 3D dataset with added Gaussian noise.

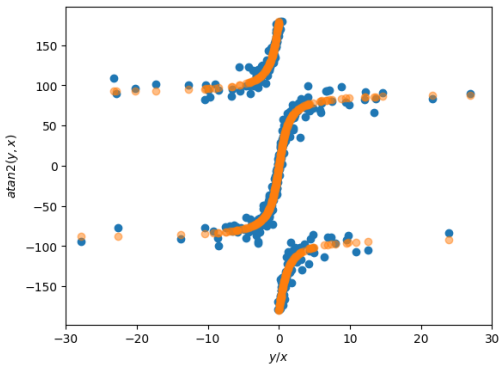
Table 5.2: Error metrics for the neural network on the 3D dataset.

5.5.2 Toy example of symbolic regression

In the 2D example, the desired equation is $\arctan 2(y, x)$, which was manually added to the PySR model as a binary operator. In both the clean and noisy case, PySR was able to generate the correct equation, demonstrating its ability to recover the mathematical relationship in both ideal and noisy conditions. Figure 5.5a shows a comparison between ground truth and predicted angles for the clean dataset, and Figure 5.5b shows the same comparison for the dataset with added noise. Note that some outliers have been removed from the figures for clarity.



(a) 2D data prediction (orange) vs. ground truth (blue) for a dataset without noise.



(b) 2D data prediction (orange) vs. ground truth (blue) for a dataset with added Gaussian nois.

Figure 5.5: 2D data prediction (orange) vs. ground truth (blue), for a 2D dataset with no noise and a dataset with added noise.

After the 2D example, the slightly more complex 3D model was tested. The angles corresponding to the dataset points were calculated according to Equations 5.1 and 5.2. These are also the desired equations, in the language of PySR $\arctan 2(y, \sqrt{x^2 + z^2})$ and $\arctan 2(x, z)$ respectively.

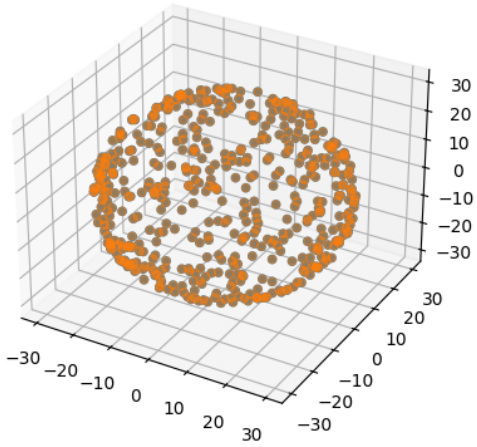
In this case, the maximum complexity had to be decreased and number of iterations had to be increased for the PySR model to find the correct equations. Through iterative fine-tuning of the model settings, PySR correctly identified the desired equations on the clean dataset. As shown in Figure 5.6a, the ground truth points and points calculated from the identified equations are perfectly aligned.

With the noisy dataset, PySR identified two potential equations for the relationship. The equations were as follows:

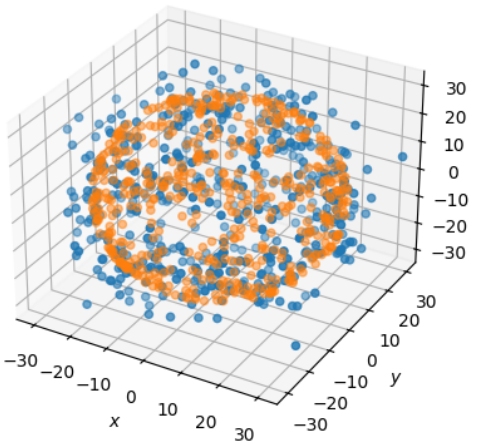
$$\theta_1 = \frac{y}{27.6824} \tag{5.3}$$

$$\theta_2 = \text{atan2}(x - 1.6847, z) \tag{5.4}$$

Figure 5.6b shows the comparison between the noisy ground truth points and the points calculated from the estimated equations. Under noisy conditions, PySR did not find the most accurate equations, but instead identified potential underlying mathematical relationships.



(a) Comparison of Ground Truth and Predicted 3D Positions



(b) Comparison of Noisy Ground Truth and Predicted 3D Positions

Figure 5.6: 3D Data Prediction (orange) vs. Ground Truth (blue), for a 3D dataset with noise.

5.6 Comparison of Methods

The purely analytical approach is highly idealized. Although, in principle, it could guarantee a mathematically exact conversion of the pose data obtained from MediaPipe, such large-scale computations are infeasible for a complex and variable humanoid robot. It also relies on the landmark coordinates from the pose estimation to be completely accurate. As such, it will not be considered for the final system.

As an option, symbolic regression was considered. This method proved feasible for the 2D example but did not find the correct equations with the noisy 3D data, meaning that it may not be suitable for noisy data with even higher dimensions. Its usefulness also diminishes in scenarios without ground truth, since the interpretability of the resulting equations becomes less meaningful. Moreover, for the purpose of this project, knowing the exact mathematical relationship is not required, but rather a reliable mapping between pose and motion.

In contrast, neural networks and deep learning methods sacrifice interpretability but are more suitable for capturing high-dimensional, nonlinear mappings directly from data. As indicated by the example results of noisy data in 2D compared to 3D, higher dimensionality could potentially reduce the impact of noise. Considering the scale and complexity of the dataset generated in this work, machine learning, particularly neural network–based models, represents the most practical and effective solution.

Another benefit of a machine learning solution in combination with the simulation system and dataset generation method discussed in previous chapters is modularity. If some part of the system was to be replaced, for example the humanoid model or the pose estimation module, one could easily generate a new dataset and train a new model.

6

Experimental Results and Discussion

This chapter presents the results of the complete system and analyzes the performance of machine learning in pose-to-motion translation. The results are then discussed, followed by suggenstions for potential improvements and future work.

6.1 Dataset Creation

With the system described in Chapter 4, a dataset was gathered that could be used to train a machine learning model. In total, a training dataset consisting of 131,280 pose-motion pairs was gathered. However, the system was not without problems. First of all, the process for creating random motions was somewhat inefficient. As new motions were generated by assigning random numbers as joint positions without taking joint limits into account, many motions were infeasible for the humanoid and failed during the planning stage. The resulting pose data was also noisy, as there was only one camera in the simulation environment which could not accurately capture a full body view of the humanoid for every motion, leading to some pose estimation results being inaccurate.

6.2 Arm motion replay

The full body dataset was fairly high-dimensional, with pose data dimensions of 99 (33 landmarks $\times$ 3 coordinates) and motion data dimensions of 47 (47 joints $\times$ 1 joint position). To evaluate the neural network and symbolic regression in a simpler case with fewer dimensions, pose and motion data specific to the landmarks and joints of the left arm were extracted, which included the left shoulder, elbow, wrist, and hand-related landmarks and joints. The following filtering steps were performed:

- Only landmarks corresponding to the left arm were selected, reducing the number of pose input dimensions to 21 (7 landmarks $\times$ 3 coordinates).
- Only joints associated with the left arm were selected, reducing the motion data dimensions from 47 to 8. These are:
  - `jC7LeftShoulder_rotx`
  - `jLeftShoulder_rotx`
  - `jLeftShoulder_roty`
  - `jLeftShoulder_rotz`

- `jLeftElbow_roty`
- `jLeftElbow_rotz`
- `jLeftWrist_rotx`
- `jLeftWrist_rotz`

6.2.1 Left arm neural network performance

The neural network model trained on the left arm dataset was the `SimpleMotionModel`. To validate the effectiveness of the data normalization method introduced in Chapter 5, another `SimpleMotionModel` was trained on the same dataset after normalization was applied. To evaluate the feasibility of the system, the trained neural network models' pose-to-motion translation was then tested on randomly selected samples from an unseen set of left arm poses and replayed on the humanoid.

6.2.1.1 Arm only neural network performance without normalization

Performance metrics for the `SimpleMotionModel` without pose normalization are presented in Figure 6.1 and Table 6.1.

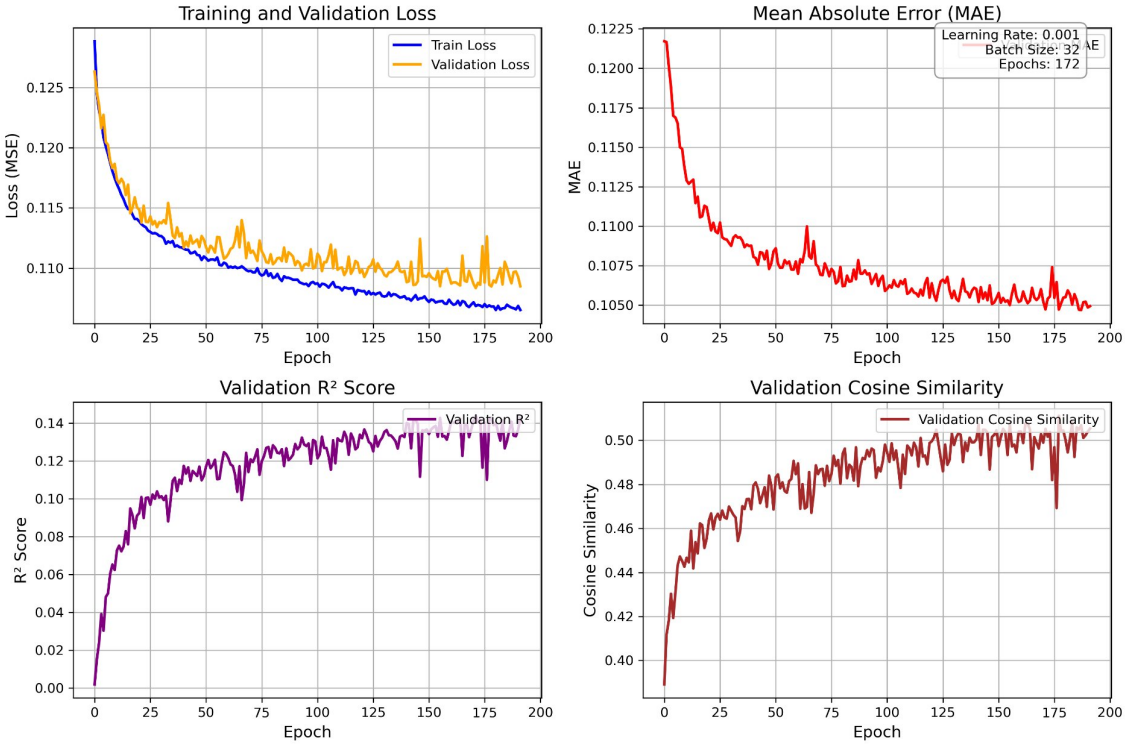


Figure 6.1: Performance metrics for the model on the left arm dataset without pose normalization.

Table 6.1: Performance metrics for the model on the left arm dataset without pose normalization.

Validation Loss	0.1108
Validation MAE	0.1056
Validation R^2 Score	0.1350
Validation Cosine Similarity	0.5010

Figure 6.2 shows a comparison between an unseen pose on the right, and the result of the motion predicted by the neural network after training on the arm only dataset without pose normalization.

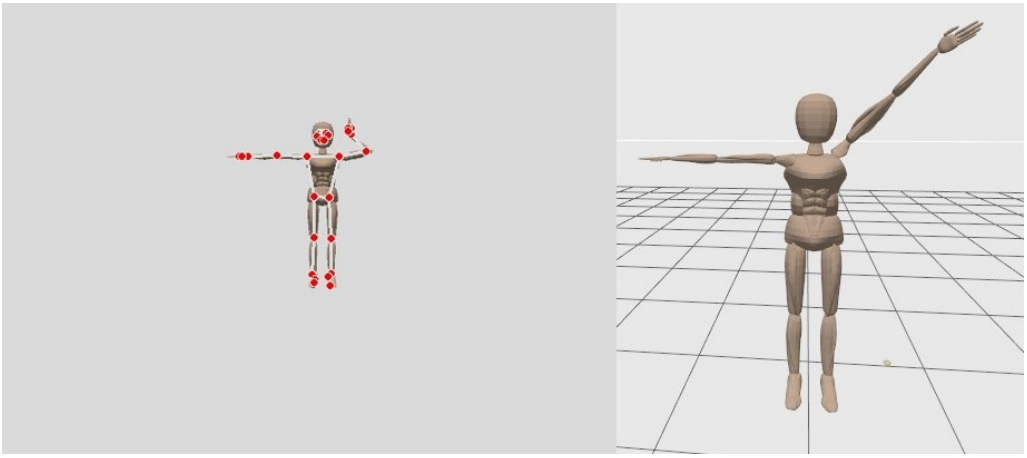


Figure 6.2: Humanoid motion predicted by the neural network without normalized training data.

6.2.1.2 Arm only neural network performance with normalization

Performance metrics for the `SimpleMotionModel` with pose normalization are presented in Figure 6.4 and Table 6.2.

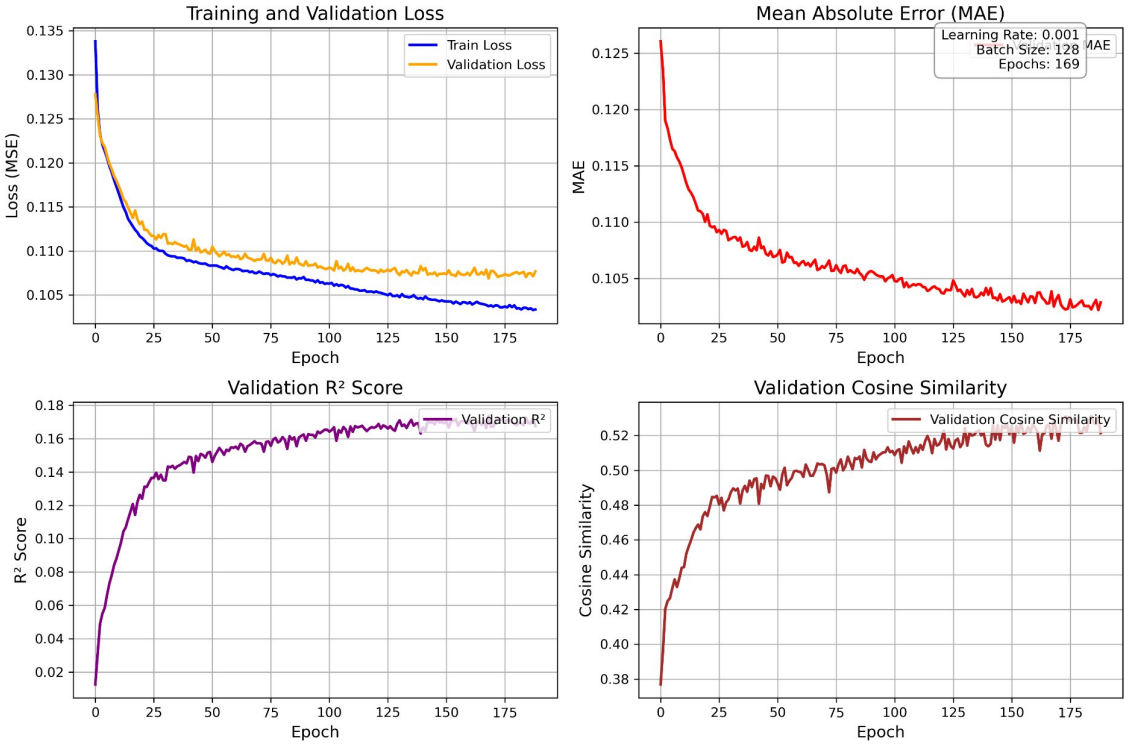


Figure 6.3: Performance metrics for the simple model on the left arm dataset with normalization.

Table 6.2: Performance metrics for the simple model on the left arm dataset with normalization.

Validation Loss	0.1081
Validation MAE	0.1025
Validation R^2 Score	0.1670
Validation Cosine Similarity	0.5210

Figure 6.4 shows a comparison between an unseen input pose and output motion predicted by the simple model after training on the arm only dataset with pose normalization.

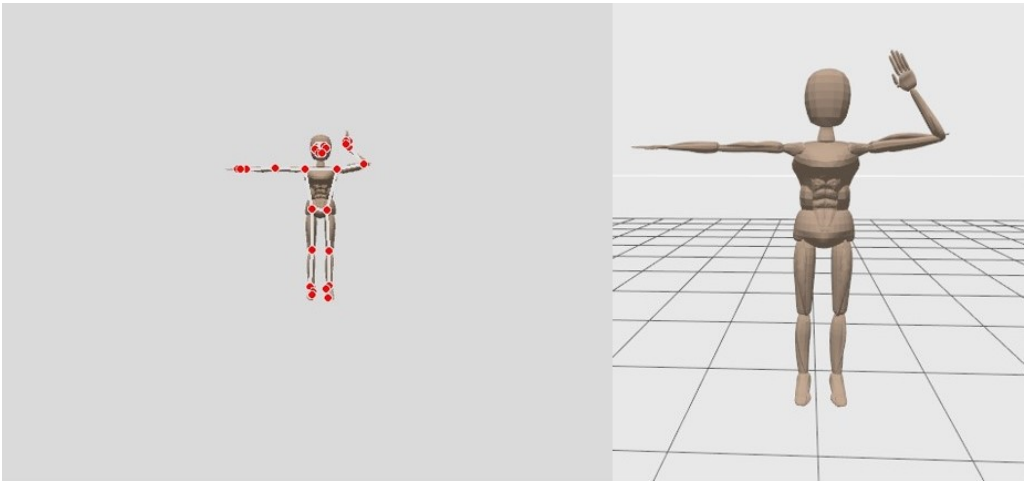


Figure 6.4: Humanoid motion predicted by the neural network with normalized training data.

Based on the observed performance metrics and comparisons from the arm only test cases, normalization seems to be effective in improving the motion prediction. Therefore, normalization was also applied in the following training experiments with full-body dataset.

6.2.2 Left arm symbolic regression performance

To compare with the neural network performance, a symbolic regression model was trained on the left arm dataset using PySR. The result was 8 equations, each corresponding to one of the filtered motion variables, which represent the relationship between the filtered pose and motion data. The equations obtained were used to calculate the predicted motion variables based on the pose data. This helped to evaluate the accuracy and reliability of the symbolic regression method by comparing the predicted motion variables with the ground truth values.

The data preprocessing described in section 5.3.1 was also used for the symbolic regression method.

The results of the symbolic regression analysis are summarized in Figure 6.5 and Table 6.3. The performance metrics for the first joint seem promising, but then rapidly decrease for the following joints. This indicates that PySR struggles to find equations that accurately describe the connection between pose and motion of the joints.

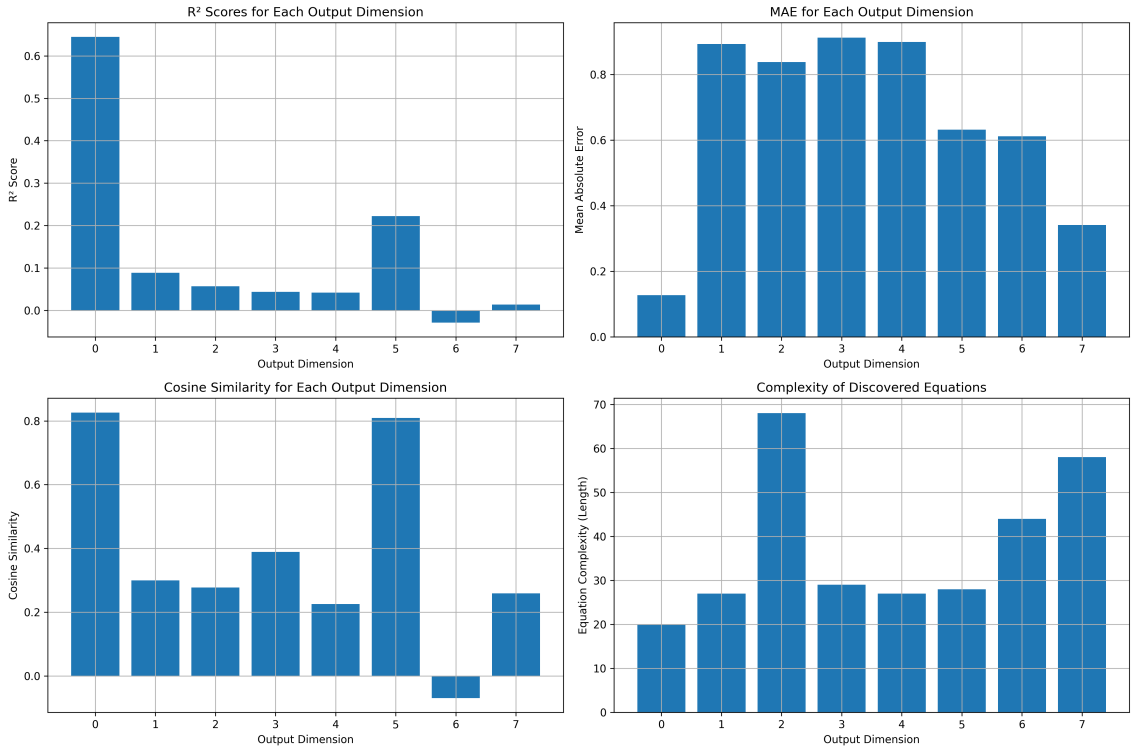


Figure 6.5: Performance metrics for symbolic regression on the left arm dataset.

Table 6.3: Performance metrics for symbolic regression on the left arm dataset.

Average R^2 Score	0.1355
Average MAE	0.6567
Average Cosine Similarity	0.3772

6.3 Full body motion replay

Following the arm only tests, neural network and symbolic regression performance was evaluated on the full dataset, containing all pose landmarks and joints. In addition to predicting motions from unseen humanoid poses, the motion prediction from real-life images to the humanoid was also tested. A video with different motions was recorded with a mobile phone camera, images were extracted from the video frames, and pose estimation data was extracted from the images using MediaPipe. The resulting pose data was given as input to the trained model to test whether the humanoid robot in Gazebo could reproduce the demonstrated poses.

6.3.1 Full body neural network performance

In order to improve both accuracy and expressiveness, training was performed on the full-body dataset using the `Improved Motion Model` neural network architecture described in Chapter 5. As in the arm only tests, one model was trained on the full

body dataset without pose normalization, and another on the same dataset with pose normalization.

6.3.1.1 Full body neural network performance without normalization

Performance metrics for the `Improved Motion Model` without pose normalization are presented in Figure 6.6 and Table 6.4.

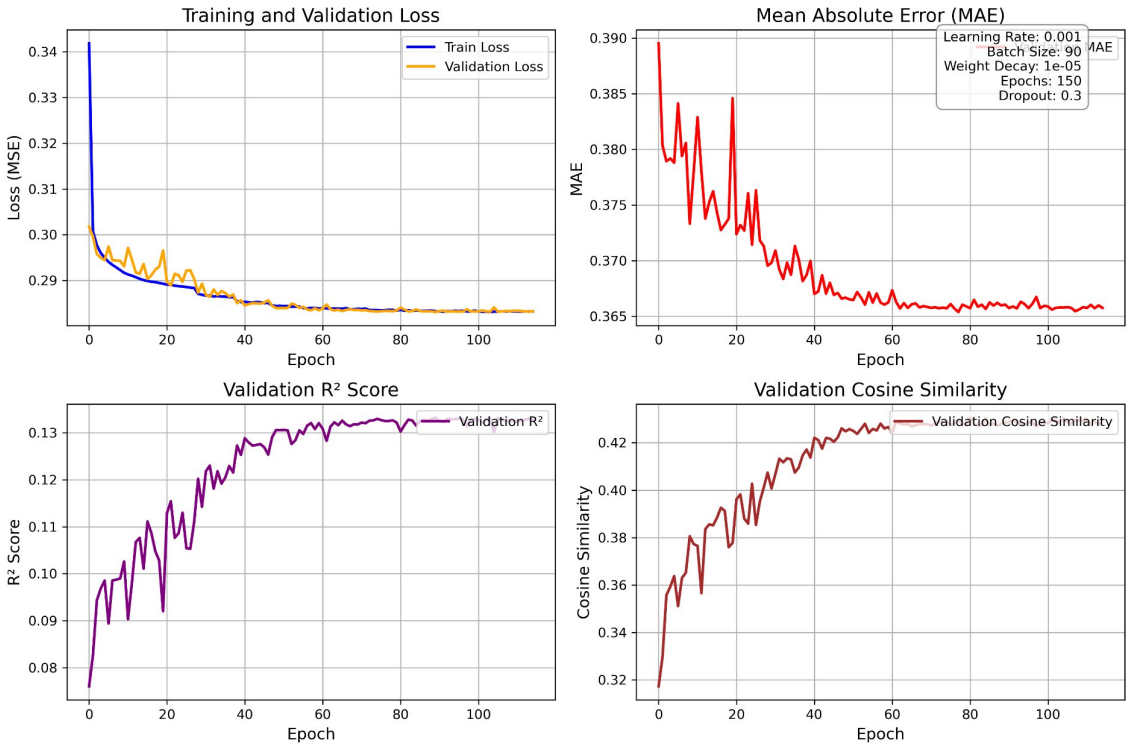


Figure 6.6: Performance metrics for the improved model on the full body dataset without normalization.

Table 6.4: Performance metrics for the improved model on the full body dataset without normalization.

Validation Loss	0.2833
Validation MAE	0.3658
Validation R^2 Score	0.1326
Validation Cosine Similarity	0.4294

To evaluate the motion predicted by the neural network on full body poses, it was used to predict humanoid motions from both simulation images and real-life images. The result of a random sample simulation pose is shown in Figure 6.7, and the result of two real-life poses are shown in Figure 6.8.

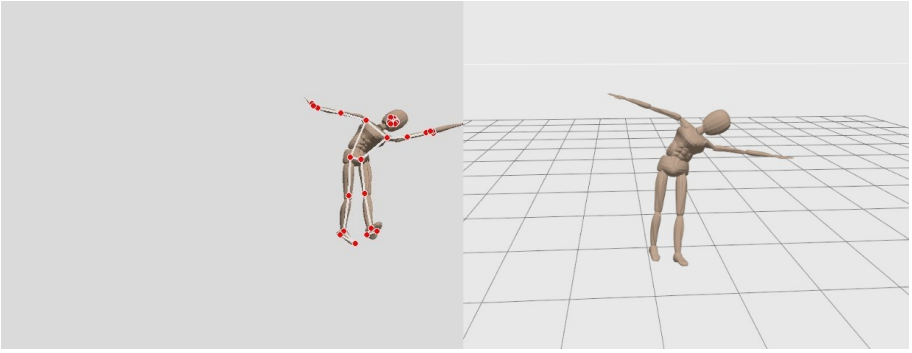


Figure 6.7: Humanoid motion predicted from simulation image by the neural network without normalized training data.

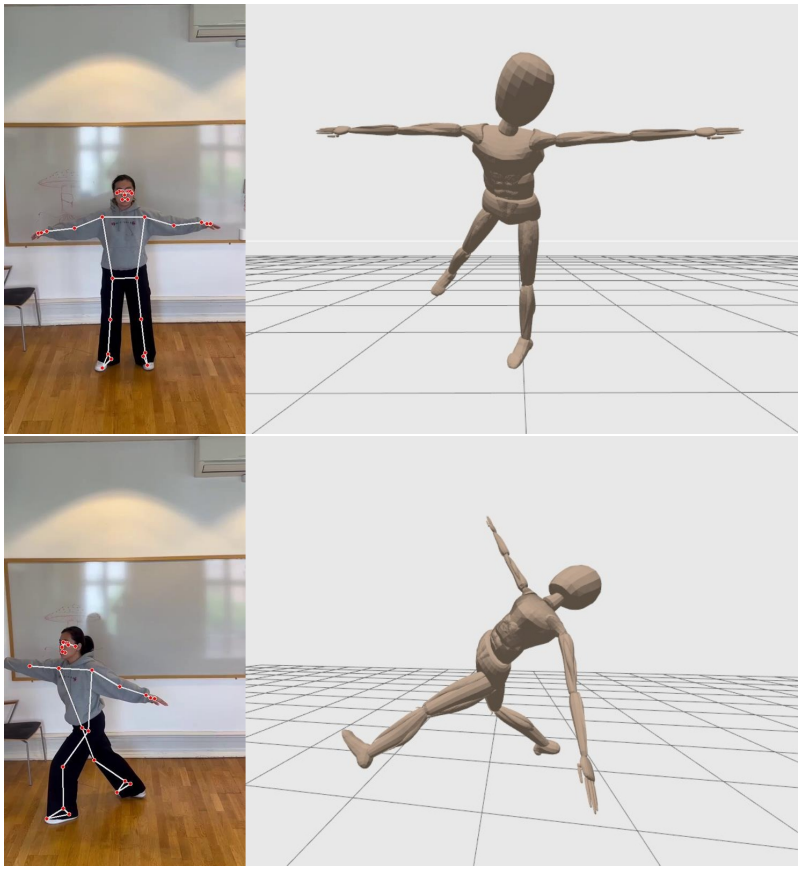


Figure 6.8: Humanoid motion predicted from real-life image by the neural network without normalized training data.

6.3.1.2 Full body neural network performance with normalization

Performance metrics for the Improved Motion Model with pose normalization are presented in Figure 6.9 and Table 6.5.

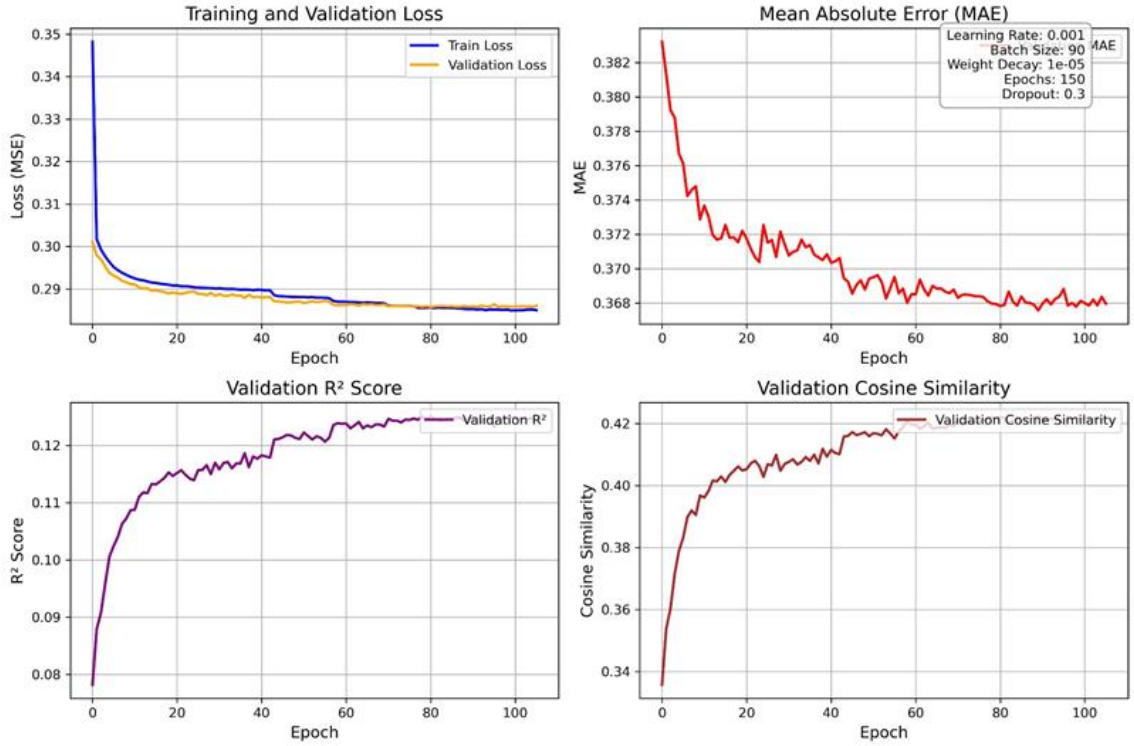


Figure 6.9: Performance metrics for the improved model on the full body dataset with normalization.

Table 6.5: Performance metrics for the improved model on the full body dataset with normalization.

Validation Loss	0.2860
Validation MAE	0.3679
Validation R^2 Score	0.1243
Validation Cosine Similarity	0.4213

As in the previous experiment, the neural network performance was evaluated by predicting humanoid motions from simulation images, see Figure 6.10, and real-life images, see Figure 6.11.

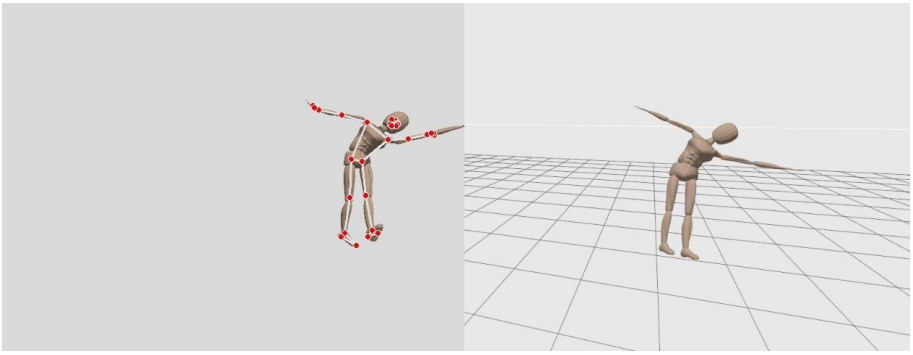


Figure 6.10: Humanoid motion predicted from simulation image by the neural network with normalized training data.

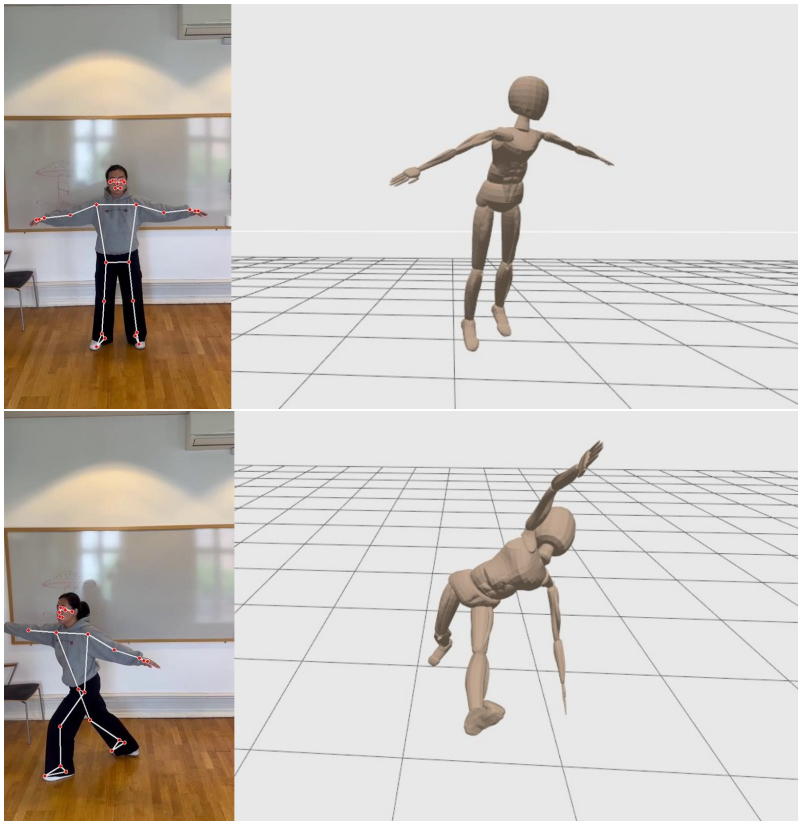


Figure 6.11: Humanoid motion predicted from real-life image by the neural network with normalized training data.

6.3.2 Full body Symbolic regression performance

Like with the arm-only dataset, a symbolic regression model was trained on the full body dataset. Using the PySR framework, mathematical equations were derived for each of the 47 joint motion variables based on the pose data input. Once the equations were generated, they were used to compute the predicted motion variables from the pose data. The predictions were then evaluated against the ground truth motion data using performance metrics.

The overall performance of the symbolic regression is summarized in Figure 6.12. Overall, they are not better than in the arm-only case, indicating that the increased complexity of the data did not improve the symbolic regression performance. The performance graphs also seem to follow a similar pattern as in the arm-only case, where the performance for the first joint in a chain is better than the following joints.

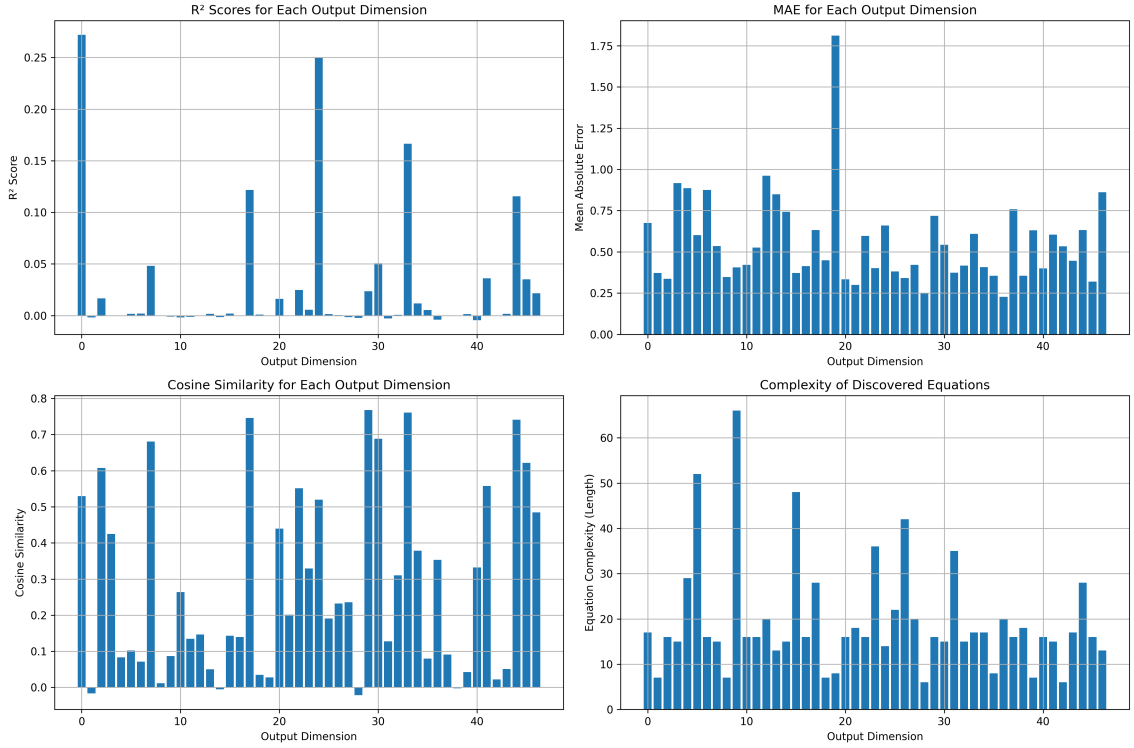


Figure 6.12: Performance metrics for symbolic regression on the full body dataset.

The key performance metrics are presented in Table 6.6. Especially the average R^2 score is significantly lower than in the arm-only case, indicating that the equations struggle to capture the variance in the motion data. This is likely due to the complexity of the kinematic relationships, the high dimensionality, and the inevitable noise of the dataset.

Table 6.6: Performance metrics for symbolic regression on the full body dataset.

Average R^2 Score	0.0257
Average MAE	0.5533
Average Cosine Similarity	0.2840

6.4 Comparison and discussion

This section discusses the main results of the the project, analyzes performance, and address issues and potential improvements of the system.

6.4.1 Dataset generation

As previously mentioned, the dataset generation system had some flaws. It reliably captured and saved pose and motion data, and the synchronization method works well, as the resulting dataset had no mismatched motion IDs. Inefficiency in motion generation and noise in the dataset likely had an impact on the machine learning performance, since more accurate training data would lead to more accurate predictions from machine learning. As a proof of concept, it was deemed good enough, improving the dataset generation system would lead to an improvement of the pose to motion translation.

6.4.2 Neural network and symbolic regression performance

The comparative experiments show that there are significant differences between symbolic regression and machine learning methods. The most promising result was with the arm-only machine learning model with normalization, which generated stable mappings between pose and motion and was able to reproduce motions accurately in simulation. Conversely, the least promising approach was the full body symbolic regression result, which was unable to fully fit the complexity of the full body kinematics.
In the introductory toy example in Chapter 5, the neural network showed promise in handling noise in higher dimensions. This was not evident when comparing arm only performance with full body performance, where the metrics were overall worse for the full body dataset. However, going from two to three dimensions with one joint angle is a significantly smaller step than the dimensional difference between the arm only and the full body dataset, and the decrease in performance was not significant enough to conclude that the theory of higher dimensionality reducing the impact of noise is necessarily incorrect.
Symbolic regression, on the other hand, struggled to find equations that would accurately map the pose data to joint positions, particularly in the full body case. While it performed well in low dimensional toy examples, its scaling was not enough to handle the entire dataset. Potentially, this was due to the high dimensionality and noise of the dataset used. This suggests that at this point in time neural networks would be the best solution for pose-to-motion translation. The high complexity of the resulting equations would also reduce the interpretability of the result, though that was not an important consideration in this case.
For the neural network models, normalization of pose data had some effect in the arm-only case for replication quality and evaluation metrics. On the full body dataset, normalization did not make a significant difference in performance. As for symbolic regression, analysis was only carried out on normalized datasets, which may have had an impact on the performance.

6.4.3 Motion replay

Motion replay from pose estimation was shown to be possible in the final system, though the resulting motions were not entirely accurate. As the best performance metrics were from the neural network models, they were used to predict motion data

to replay on the humanoid from unseen pose data. It is important to note that in both the simpler arm only and the full body case, many motions failed during the planning or execution stage, handled by MoveIt 2. While it is difficult to analyze exactly what went wrong, as the error prevented the motion from being executed as all, it was most likely due to the predicted motion being infeasible, either due to being outside joint limits or resulting in a self-collision for the humanoid. In other words, the predicted motion was incorrect in many cases, as the input pose was executable when captured from simulation or performed by a human.

6.4.4 Future research and potential improvements

One way of improving the quality of the training dataset would be to use multiple cameras to capture the pose of the humanoid from different angles. An issue with the generated dataset was that the humanoid would sometimes be positioned in a way where the body position was not visible to the camera, for example being turned to the side or leaning away. This leads to inefficiency in the dataset generation system, as it simply discards the motion and generates a new one if the pose is not detected, or dataset noise from inaccurate pose detections that do not match the actual pose. Having more camera angles would likely decrease the rate of discarded motions, and increase accuracy of the detected poses.
The dataset generation efficiency could also potentially be improved by using the official MoveIt 2 plugin for motion generation, which was not available for ROS 2 Humble at the time of this project. The MoveIt 2 plugin has the function to move to a "random valid" position, which would reduce the rate of invalid motions generated by simply assigning random values to joints, as in this project. However, using the official plugin would require either updating the ROS 2 version of the entire project or the plugin being backported to Humble.
Regarding machine learning performance, a potential direction could be the integration of symbolic regression and neural networks to improve the motion prediction. Recent research in neural-guided symbolic regression, for example, has proposed that deep models might be used to lead an inductive search for symbolic expressions [17]. Additionally, the use of automated machine learning (AutoML) frameworks such as AutoGluon [33] could be explored to accelerate model selection and hyperparameter tuning.
Another promising path for improvement could be to include feature engineering in the machine learning pipeline. In the current approach, the model directly maps the landmark (x, y, z) coordinates to joint states, resulting in a high-dimensional and computationally expensive regression problem. In contrast, if measures of landmark angles or relative directions were calculated and then fed to the model, it could effectively reduce input dimensionality. This transformation might help reduce training time and increase prediction accuracy.

7
Conclusion

The main objective of this thesis was to investigate the feasibility of translating human poses into executable motions for a humanoid robot by translating pose detection results to joint commands. Over the course of the project, a complete pipeline to generate a training dataset, train a machine learning model, and perform motion replay was developed to achieve this objective. The resulting system shows that translating pose estimation to humanoid motion commands is possible, but the result is not very accurate in the current system.
A key benefit of the system is that it does not rely on pre-existing training datasets or specialized equipment, such as motion capture equipment or depth cameras. Relying on 3D pose estimation from RGB camera images does however negatively impact the accuracy of the detected poses.
The work began by examining how motion commands could be represented and stored as executable files, followed by the successful construction of a simulated humanoid robot within the Gazebo environment.
Subsequently, the feasibility of pose-to-motion mapping was explored through different methodological perspectives, including analytical methods, symbolic regression, and machine learning approaches. This comparative analysis demonstrated the limitations of purely analytical solutions when applied to large-scale and noisy datasets, while highlighting the potential of machine learning models.
Finally, by constructing a dataset of synchronized pose–motion pairs and training both neural network and symbolic regression models and comparing their performance, neural network models were determined to be the best solution for pose-to-motion translation. By validating the neural network performance with both simulation and real-world poses, the thesis established that neural networks offer a practical and effective approach for pose-to-motion translation. Thus, the project successfully advanced from conceptual exploration to the implementation of a full end-to-end pipeline.

7.1 Ethics

Although this project does not involve surveillance, a potential use case for the system is to detect safety incidents to improve workplace safety. According to the Swedish Authority for Privacy Protection (IMY), camera surveillance entails an infringement of privacy and requires a strong reason to be justified [34]. While the purpose is not to develop a system that can identify individuals, collect personal data, or be used for general surveillance, using pose estimation could potentially be

used for monitoring workers, for example by detecting if a person walks away from their station.
From a social sustainability perspective, workplace safety monitoring systems directly contribute to workers' physical and mental well-being. Recent studies have highlighted the psychological distress caused by occupational injuries among industrial workers, which significantly impacts their work performance and quality of life [35]. AI-enabled safety monitoring prevents injuries through early detection and reduces safety personnel workload. This allows a focus on preventive strategies, improving workplace satisfaction and safety culture.
The economic aspects are equally important, particularly for smaller companies with limited resources. Traditional safety infrastructure often requires substantial initial investment, which can be a barrier for many businesses. Our simulation-based approach reduces both testing costs and deployment risks, making advanced safety solutions more accessible to companies of all sizes.

Bibliography

[1] Infotiv, *Simlan, simulation for indoor multi-camera localization and navigation (1.0.4)*. [Online]. Available: https://github.com/infotiv-research/SIMLAN.

[2] L. Penco, B. Clement, V. Modugno, *et al.*, "Robust real-time whole-body motion retargeting from human to humanoid", in *2018 IEEE-RAS 18th International Conference on Humanoid Robots (Humanoids)*, 2018, pp. 425–432. DOI: 10.1109/HUMANOIDS.2018.8624943.

[3] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang, *Expressive whole-body control for humanoid robots*, 2024. arXiv: 2402.16796 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2402.16796.

[4] Y. Ze, Z. Chen, J. P. Araújo, *et al.*, *Twist: Teleoperated whole-body imitation system*, 2025. arXiv: 2505.02833 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2505.02833.

[5] C. Stanton, A. Bogdanovych, and E. Ratanasena, "Teleoperation of a humanoid robot using full-body motion capture, example movements, and machine learning", Dec. 2012.

[6] T. He, Z. Luo, W. Xiao, *et al.*, "Learning human-to-humanoid real-time whole-body teleoperation", in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 8944–8951. DOI: 10.1109/IROS58592.2024.10801984.

[7] T. He, Z. Luo, X. He, *et al.*, *Omnih2o: Universal and dexterous human-to-humanoid whole-body teleoperation and learning*, 2024. arXiv: 2406.08858 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2406.08858.

[8] Z. Fu, Q. Zhao, Q. Wu, G. Wetzstein, and C. Finn, *Humanplus: Humanoid shadowing and imitation from humans*, 2024. arXiv: 2406.10454 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2406.10454.

[9] L. Fortini, M. Leonori, J. M. Gandarias, and A. Ajoudani, *Open-vico: An open-source gazebo toolkit for multi-camera-based skeleton tracking in human-robot collaboration*, 2022. DOI: 10.48550/ARXIV.2203.14733. [Online]. Available: https://arxiv.org/abs/2203.14733.

[10] A. Sripada, H. Asokan, A. Warrier, *et al.*, "Teleoperation of a humanoid robot with motion imitation and legged locomotion", in *2018 3rd International Conference on Advanced Robotics and Mechatronics (ICARM)*, 2018, pp. 375–379. DOI: 10.1109/ICARM.2018.8610719.

[11] S. Baek, A. Kim, J.-Y. Choi, E. Ha, and J.-W. Kim, "Human motion retargeting to a full-scale humanoid robot using a monocular camera and human pose estimation", *International Journal of Control, Automation and Systems*, vol. 22, pp. 2860–2870, Sep. 2024. DOI: 10.1007/s12555-023-0686-y.

[12] H. Khalil, E. Coronado, and G. Venture, "Human motion retargeting to pepper humanoid robot from uncalibrated videos using human pose estimation", in *2021 30th IEEE International Conference on Robot Human Interactive Communication (RO-MAN)*, 2021, pp. 1145–1152. DOI: 10.1109/RO-MAN50785.2021.9515495.

[13] V. Nair and G. E. Hinton, "Rectified linear units improve restricted Boltzmann machines", in *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 2010, pp. 807–814.

[14] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)", *arXiv e-prints*, arXiv:1606.08415, arXiv:1606.08415, Jun. 2016. DOI: 10.48550/arXiv.1606.08415. arXiv: 1606.08415 [cs.LG].

[15] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors", *Nature*, vol. 323, no. 6088, pp. 533–536, Oct. 1986, ISSN: 1476-4687. DOI: 10.1038/323533a0. [Online]. Available: https://doi.org/10.1038/323533a0.

[16] N. R. Draper and H. Smith, *Applied Regression Analysis*. Wiley, 1998.

[17] N. Makke and S. Chawla, "Interpretable scientific discovery with symbolic regression: A review", *Artificial Intelligence Review*, vol. 57, no. 2, Jan. 2024. DOI: 10.1007/s10462-023-10622-0.

[18] J. R. Koza, "Genetic programming as a means for programming computers by natural selection", *Statistics and Computing*, vol. 4, pp. 87–112, Jun. 1994.

[19] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild", *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074. [Online]. Available: https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

[20] Open Robotics, *Gazebo*. [Online]. Available: https://gazebosim.org/home.

[21] D. Coleman, I. A. Șucan, S. Chitta, and N. Correll, "Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study", *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, May 2014. DOI: 10.6092/JOSER_2014_05_01_p3.

[22] I. A. Sucan and S. Chitta, *Moveit*. [Online]. Available: https://moveit.ai/.

[23] A. Orsula, *Pymoveit2*, 2025. [Online]. Available: https://github.com/AndrejOrsula/pymoveit2.

[24] P. D. Fagan, *MoveIt 2 Python Library: A Software Library for Robotics Education and Research*, 2023. [Online]. Available: https://github.com/moveit/moveit2/tree/main/moveit_py.

[25] C. Zheng, W. Wu, C. Chen, *et al.*, *Deep learning-based human pose estimation: A survey*, 2023. arXiv: 2012.13392 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2012.13392.

[26] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "Openpose: Realtime multi-person 2d pose estimation using part affinity fields", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 172–186, 2021. DOI: 10.1109/TPAMI.2019.2929257.

[27] MMPose Contributors, *OpenMMLab Pose Estimation Toolbox and Benchmark*, https://github.com/open-mmlab/mmpose, 2020.

[28] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, and M. Grundmann, *Blazepose: On-device real-time body pose tracking*, 2020. arXiv: 2006.10204 [cs.CV]. [Online]. Available: https://arxiv.org/abs/2006.10204.

[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition", in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[30] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift", *arXiv e-prints*, arXiv:1502.03167, arXiv:1502.03167, Feb. 2015. DOI: 10.48550/arXiv.1502.03167. arXiv: 1502.03167 [cs.LG].

[31] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks", *Journal of Machine Learning Research - Proceedings Track*, vol. 9, pp. 249–256, Jan. 2010.

[32] M. Cranmer, *Interpretable machine learning for science with pysr and symbolicregression.jl*, 2023. arXiv: 2305.01582 [astro-ph.IM]. [Online]. Available: https://arxiv.org/abs/2305.01582.

[33] N. Erickson, J. Mueller, A. Shirkov, *et al.*, "AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data", *arXiv e-prints*, arXiv:2003.06505, arXiv:2003.06505, Mar. 2020. DOI: 10.48550/arXiv.2003.06505. arXiv: 2003.06505 [stat.ML].

[34] Swedish Authority for Privacy Protection, *Video-surveillance of employees*. [Online]. Available: https://www.imy.se/en/organisations/camera-surveillance/videosurveillance-of-employees (visited on 12/17/2024).

[35] S. Kim, M. Kang, and J. Park, "Digital industrial accidents: A case study of the mental distress of platform workers in South Korea", *International Journal of Social Welfare*, vol. 31, no. 3, pp. 355–367, 2022. DOI: https://doi.org/10.1111/ijsw.12522.

A

Appendix 1

Listing A.1: Pose Normalization Code

```
def data_preprocess(pose_dataset):
    landmark_index = 23
    normalized = pose_dataset - np.tile(
        pose_dataset[:, landmark_index*3:landmark_index*3 + 3], (1, 33)
    )
    return normalized
```

Listing A.2: Simple Motion Model Code

```
class SimpleMotionModel(nn.Module):
    def __init__(self, input_size, output_size):
        super(SimpleMotionModel, self).__init__()
        self.model = nn.Sequential(
            nn.Linear(input_size, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, output_size)
        )
    def forward(self, x):
        return self.model(x)
```

Listing A.3: Improved Motion Model Code

```
class ImprovedMotionModel(nn.Module):
    def __init__(self, input_size, output_size, hidden_sizes=[124, 64, 32
        super(ImprovedMotionModel, self).__init__()
        self.input_layer = nn.Linear(input_size, hidden_sizes[0])
        self.input_bn = nn.BatchNorm1d(hidden_sizes[0])
        self.hidden_layers = nn.ModuleList()
        self.residual_projections = nn.ModuleList()
        for i in range(len(hidden_sizes) - 1):
            layer = nn.Sequential(
                nn.Linear(hidden_sizes[i], hidden_sizes[i + 1]),
                nn.BatchNorm1d(hidden_sizes[i + 1]),
                nn.GELU(),
                nn.Dropout(dropout)
```

```
            )
            self.hidden_layers.append(layer)
            if hidden_sizes[i] != hidden_sizes[i + 1]:
                projection = nn.Linear(hidden_sizes[i], hidden_sizes[i +
            else:
                projection = nn.Identity()
            self.residual_projections.append(projection)
        self.output_layer = nn.Linear(hidden_sizes[-1], output_size)
        self.apply(self._init_weights)
    def _init_weights(self, m):
        if isinstance(m, nn.Linear):
            nn.init.xavier_uniform_(m.weight)
            if m.bias is not None:
                nn.init.zeros_(m.bias)
    def forward(self, x):
        x = self.input_layer(x)
        x = self.input_bn(x)
        x = torch.nn.functional.gelu(x)
        for i, layer in enumerate(self.hidden_layers):
            residual = x
            x = layer(x)
            projected_residual = self.residual_projections[i](residual)
            x = x + projected_residual
        return self.output_layer(x)
```

Listing A.4: Training Loop Code

```
for epoch in range(1, num_epochs + 1):
    train_loss = train_one_epoch(model, train_loader, criterion, optimize
    val_loss, val_mae, val_cosine_similarity, val_r2 = evaluate(model, va
    scheduler.step(val_loss)
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), model_save_path)
    else:
        patience_counter += 1
        if patience_counter >= patience:
            break
```

DEPARTMENT OF SOME SUBJECT OR TECHNOLOGY
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden
www.chalmers.se

CHALMERS
UNIVERSITY OF TECHNOLOGY